Datasets & Preprocessing Reference

LP IV Final Problem Statements 26-27 - preprocessing grouped by dataset. Comments show per-PS variants when the same dataset is reused differently.

Dataset	Pre-split?	Used in	Preprocessing

Hand-written MNIST	Yes (2 CSVs)	1A, 2B	`train = pd.read_csv("mnist_train.csv"); test = pd.read_csv("mnist_test.csv")`
			`x_train = train.drop(columns="label").to_numpy(); y_train = train["label"].to_numpy()`
			`# 1A FNN:  x_train = x_train / 255.0  # shape (-1, 784)`
			`# 2B CNN:  x_train = x_train.reshape(-1,28,28,1) / 255.0`

Fashion MNIST	Yes (2 CSVs)	1B, 5C	`train = pd.read_csv("fashion-mnist_train.csv"); test = pd.read_csv("fashion-mnist_test.csv")`
			`x_train = train.drop(columns="label").to_numpy(); y_train = train["label"].to_numpy()`
			`# 1B FNN:  x_train = x_train / 255.0  # shape (-1, 784)`
			`# 5C Transfer:  x = x_train.reshape(-1,28,28,1)[:10000]  # subsample`
			`#  layers.Resizing(32,32), layers.Concatenate()([x,x,x]), layers.Rescaling(1/127.5, offset=-1)`

CIFAR10	No (single CSV)	1C, 2A	`df = pd.read_csv("cifar-10.csv")`
			`y = df["label"].to_numpy(); x = df.drop(columns="label").to_numpy()`
			`x_tr, x_te, y_tr, y_te = train_test_split(x, y, test_size=0.2, stratify=y, random_state=42)`
			`# 1C FNN:  shape (-1, 3072); /255 inline via layers.Rescaling(1/255)`
			`# 2A CNN:  x = x.reshape(-1, 32, 32, 3); /255`

Sign Language MNIST	Yes (2 CSVs)	1E, 2C, 3F	`train = pd.read_csv("sign_mnist_train.csv"); test = pd.read_csv("sign_mnist_test.csv")`
			`# labels are 0-24 (J=9 missing), so num_classes = 25`
			`# 1E FNN:  x = x / 255; shape (-1, 784)`
			`# 2C CNN:  x = x.reshape(-1, 28, 28, 1) / 255`
			`# 3F AE anomaly:`
			`#  df = pd.concat([train, test])`
			`#  y_anom = (df["label"] == 0).astype(int); x = df.drop(columns="label") / 255`
			`#  x_normal = x[y_anom == 0]  # AE trained on 23 classes`

Credit Card Fraud	No	3A	`df = pd.read_csv("creditcard.csv")`
			`y = df["Class"].to_numpy()`
			`x = df.drop(columns=["Class", "Time"]).to_numpy()`
			`x = StandardScaler().fit_transform(x)`
			`x_normal = x[y == 0]  # AE trained only on non-fraud`
			`# threshold = np.quantile(err[y == 0], 0.99)`

ECG5000	No (no header)	3B, 3E (substitute for MVTec / NSL-KDD)	`df = pd.read_csv("ecg.csv", header=None)`
			`x = df.iloc[:, :-1].to_numpy().astype("float32")`
			`y = (df.iloc[:, -1] != 1).astype(int)  # label 1 = normal, else anomaly`
			`x = (x - x.min()) / (x.max() - x.min())`
			`x_normal = x[y == 0]`

Energy Consumption	No	3C	`df = pd.read_csv("energy consumption.csv").dropna()`
			`y = df["Anomaly_Label"].to_numpy().astype(int)`
			`x = df.drop(columns=["Timestamp", "Anomaly_Label"]).to_numpy()`
			`x = MinMaxScaler().fit_transform(x)`
			`# windows (W=10) for LSTM-AE`
			`Xw = np.stack([x[i:i+10] for i in range(len(x) - 9)])`
			`yw = np.stack([y[i:i+10] for i in range(len(y) - 9)]).max(axis=1)`
			`Xw_norm = Xw[yw == 0]`

Dataset	Pre-split?	Used in	Preprocessing

NASA Turbofan	No (label derived)	3D	`df = pd.read_csv("NASA TurboFan Engine.csv")`
			`df["max_cycle"] = df.groupby("unit")["cycle"].transform("max")`
			`df["healthy"] = df["cycle"] <= df["max_cycle"] * 0.2    # first 20% cycles`
			`feat = [c for c in df.columns if c.startswith(("sensor", "setting"))]`
			`x = MinMaxScaler().fit_transform(df[feat])`
			`Xw = np.stack([x[i:i+15] for i in range(len(x) - 14)])  # W=15`

IMDB Reviews	No	4A	`df = pd.read_csv("IMDB Dataset.csv").sample(10000, random_state=42)`
			`y = (df["sentiment"] == "positive").astype(int).to_numpy()`
			`sentences = [re.findall(r"[a-z]+", r.lower()) for r in df["review"]]`
			`w2v = Word2Vec(sentences, vector_size=50, window=5, min_count=2, sg=0, epochs=5)`
			`X = np.stack([np.mean([w2v.wv[t] for t in s if t in w2v.wv] or [np.zeros(50)], axis=0)`
			`for s in sentences])`
			`X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)`

Twitter	No	4B, 4D	`df = pd.read_csv("twitter_dataset.csv")`
			`sentences = [re.findall(r"[a-z]+", str(t).lower()) for t in df["Text"].dropna()]`
			`w2v = Word2Vec(sentences, vector_size=50, window=5, min_count=2, sg=0, epochs=5)`
			`# 4B: w2v.wv.most_similar("love")`
			`# 4D: doc_vecs = np.stack([mean embedding per tweet])`
			`#     cosine_similarity(query_vec, doc_vecs)`

document3.txt	N/A (tiny corpus)	4C	`text = open("document3.txt").read()`
			`sentences = [re.findall(r"[a-z]+", s.lower()) for s in text.split(".") if s.strip()]`
			`w2v = Word2Vec(sentences, vector_size=50, window=3, min_count=1, sg=0, epochs=50)`

Caltech-101	No (folders/class)	5A	`train = keras.utils.image_dataset_from_directory("caltech-101",`
			`image_size=(128,128), batch_size=32, validation_split=0.2, subset="training", seed=42)`
			`val = keras.utils.image_dataset_from_directory("caltech-101",`
			`image_size=(128,128), batch_size=32, validation_split=0.2, subset="validation", seed=42)`
			`# model: VGG16(include_top=False, weights="imagenet") frozen, Rescaling(1/255), Dense head`

Intel Image	No (folders/class)	5B	`train = keras.utils.image_dataset_from_directory("Intel Image Classification",`
			`image_size=(128,128), batch_size=32, validation_split=0.2, subset="training", seed=42)`
			`# model: MobileNetV2(include_top=False, weights="imagenet") frozen`
			`#   layers.Rescaling(1/127.5, offset=-1)   # MobileNet expects [-1, 1]`

1A. FNN - Hand-written MNIST

FNN baseline.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/HandWritten MNIST/mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/HandWritten MNIST/mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy()
x_test = test.drop(columns="label").to_numpy()

print(x_train.shape, x_test.shape)
```

```
(60000, 784) (10000, 784)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Rescaling(1./255),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

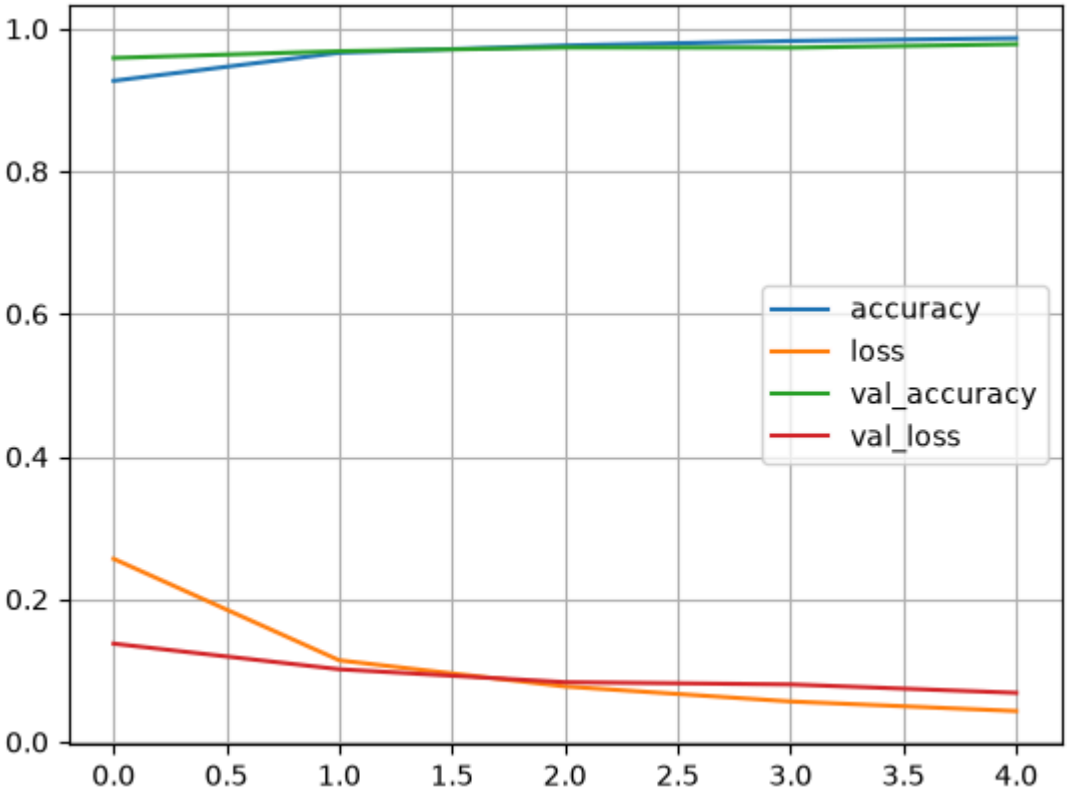
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 878us/step - accuracy: 0.9269 - loss: 0.2567 - val_accuracy: 0.9589 - val_loss:
0.1377
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 825us/step - accuracy: 0.9662 - loss: 0.1143 - val_accuracy: 0.9684 - val_loss:
0.1018
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 820us/step - accuracy: 0.9764 - loss: 0.0776 - val_accuracy: 0.9735 - val_loss:
0.0836
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 818us/step - accuracy: 0.9827 - loss: 0.0566 - val_accuracy: 0.9733 - val_loss:
0.0806
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 819us/step - accuracy: 0.9866 - loss: 0.0432 - val_accuracy: 0.9782 - val_loss:
0.0687
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
  1/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9688 - loss: 0.0675
118/313 ━━━━━━━━━━━━━━━━━━━━ 0s 430us/step - accuracy: 0.9693 - loss: 0.0955
238/313 ━━━━━━━━━━━━━━━━━━━━ 0s 424us/step - accuracy: 0.9764 - loss: 0.0734
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 442us/step - accuracy: 0.9782 - loss: 0.0687
test loss: 0.0687  test accuracy: 0.9782
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



1B. FNN - Fashion MNIST

FNN baseline.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/Fashion MNIST/fashion-mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/Fashion MNIST/fashion-mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy()
x_test = test.drop(columns="label").to_numpy()

print(x_train.shape, x_test.shape)
```

```
(60000, 784) (10000, 784)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Rescaling(1./255),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

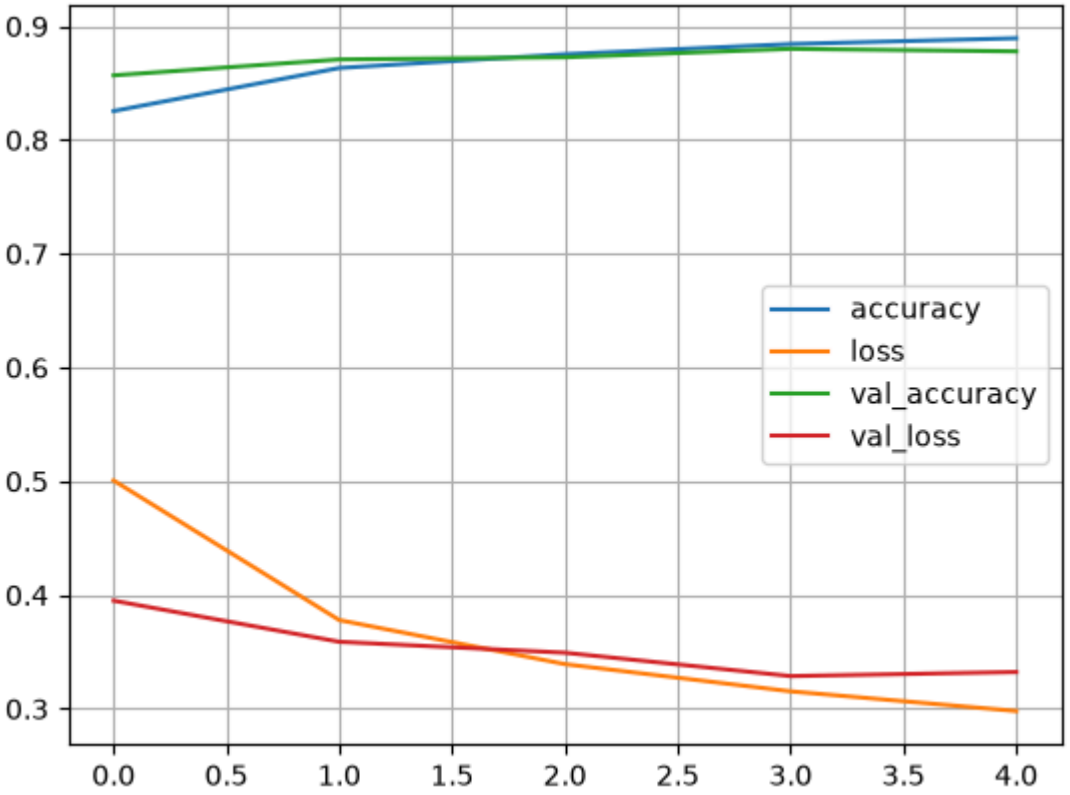
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 877us/step - accuracy: 0.8254 - loss: 0.5006 - val_accuracy: 0.8568 - val_loss:
0.3950
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 821us/step - accuracy: 0.8633 - loss: 0.3779 - val_accuracy: 0.8708 - val_loss:
0.3589
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 831us/step - accuracy: 0.8754 - loss: 0.3392 - val_accuracy: 0.8727 - val_loss:
0.3492
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 846us/step - accuracy: 0.8843 - loss: 0.3153 - val_accuracy: 0.8801 - val_loss:
0.3287
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 820us/step - accuracy: 0.8894 - loss: 0.2980 - val_accuracy: 0.8779 - val_loss:
0.3323
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
  1/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.8750 - loss: 0.3918
117/313 ━━━━━━━━━━━━━━━━━━━━ 0s 432us/step - accuracy: 0.8745 - loss: 0.3384
236/313 ━━━━━━━━━━━━━━━━━━━━ 0s 426us/step - accuracy: 0.8750 - loss: 0.3400
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 445us/step - accuracy: 0.8779 - loss: 0.3323
test loss: 0.3323  test accuracy: 0.8779
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



1C. FNN - CIFAR10

Single CSV, so we split it ourselves. FNN on RGB caps around 45-50%.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
```

In [2]:
```
df = pd.read_csv("../datasets/FNN and CNN/CIFAR10/cifar-10.csv")
y = df["label"].to_numpy()
x = df.drop(columns="label").to_numpy()
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42, stratify=y)
print(x_train.shape, x_test.shape)
```

```
(40000, 3072) (10000, 3072)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(3072,)),
    layers.Rescaling(1./255),
    layers.Dense(256, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 3072)	0
dense (Dense)	(None, 256)	786,688
dense_1 (Dense)	(None, 10)	2,570

Total params: 789,258 (3.01 MB)

Trainable params: 789,258 (3.01 MB)

Non-trainable params: 0 (0.00 B)

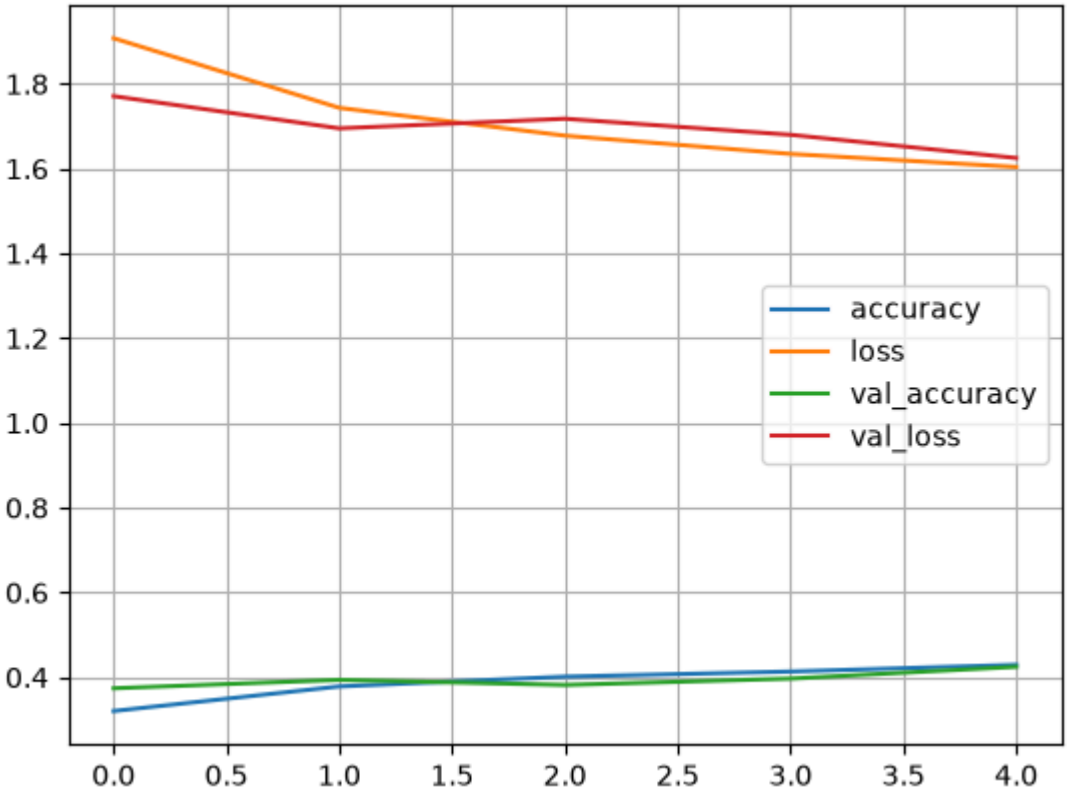
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

```
Epoch 1/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.3201 - loss: 1.9076 - val_accuracy: 0.3739 - val_loss:
1.7710
Epoch 2/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.3782 - loss: 1.7434 - val_accuracy: 0.3937 - val_loss:
1.6950
Epoch 3/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4013 - loss: 1.6778 - val_accuracy: 0.3817 - val_loss:
1.7177
Epoch 4/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4133 - loss: 1.6347 - val_accuracy: 0.3967 - val_loss:
1.6798
Epoch 5/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.4292 - loss: 1.6038 - val_accuracy: 0.4245 - val_loss:
1.6251
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
  1/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.4375 - loss: 1.5733
 88/313 ━━━━━━━━━━━━━━━━━━━━ 0s 581us/step - accuracy: 0.4354 - loss: 1.5952
171/313 ━━━━━━━━━━━━━━━━━━━━ 0s 594us/step - accuracy: 0.4311 - loss: 1.6064
254/313 ━━━━━━━━━━━━━━━━━━━━ 0s 598us/step - accuracy: 0.4275 - loss: 1.6215
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 619us/step - accuracy: 0.4245 - loss: 1.6251
test loss: 1.6251  test accuracy: 0.4245
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



1E. FNN - Sign Language MNIST

FNN baseline.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy()
x_test = test.drop(columns="label").to_numpy()

print(x_train.shape, x_test.shape)
```

```
(27455, 784) (7172, 784)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Rescaling(1./255),
    layers.Dense(128, activation="relu"),
    layers.Dense(25, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 25)	3,225

Total params: 103,705 (405.10 KB)

Trainable params: 103,705 (405.10 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

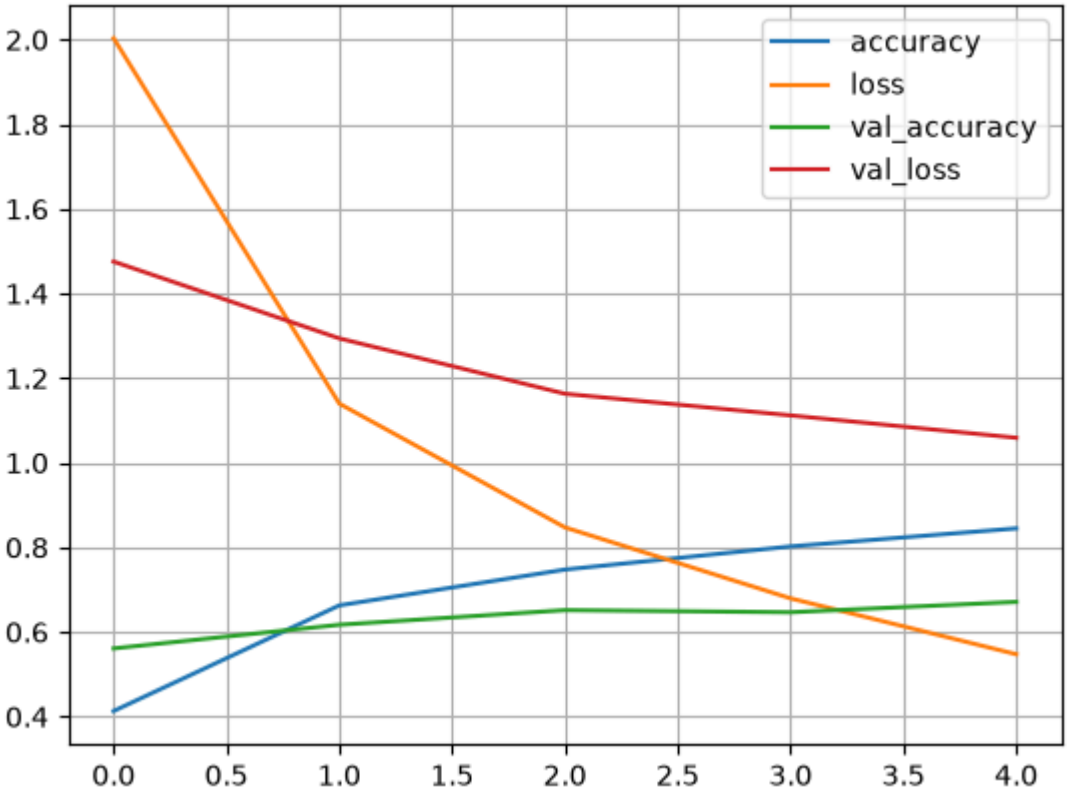
```
Epoch 1/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.4130 - loss: 2.0034 - val_accuracy: 0.5611 - val_loss: 1.4755
Epoch 2/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 1s 910us/step - accuracy: 0.6628 - loss: 1.1397 - val_accuracy: 0.6170 - val_loss:
1.2940
Epoch 3/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 1s 908us/step - accuracy: 0.7474 - loss: 0.8471 - val_accuracy: 0.6516 - val_loss:
1.1631
Epoch 4/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 1s 904us/step - accuracy: 0.8021 - loss: 0.6793 - val_accuracy: 0.6465 - val_loss:
1.1119
Epoch 5/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 1s 907us/step - accuracy: 0.8448 - loss: 0.5473 - val_accuracy: 0.6711 - val_loss:
1.0590
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
  1/225 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7812 - loss: 0.5648
115/225 ━━━━━━━━━━━━━━━━━━━━ 0s 440us/step - accuracy: 0.6655 - loss: 1.0864
225/225 ━━━━━━━━━━━━━━━━━━━━ 0s 460us/step - accuracy: 0.6711 - loss: 1.0590
test loss: 1.0590  test accuracy: 0.6711
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
```

```
plt.grid()
plt.show()
```



2A. CNN - CIFAR10

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
```

In [2]:
```
df = pd.read_csv("../datasets/FNN and CNN/CIFAR10/cifar-10.csv")
y = df["label"].to_numpy()
x = df.drop(columns="label").to_numpy().reshape(-1, 32, 32, 3)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42, stratify=y)
print(x_train.shape, x_test.shape)
```

```
(40000, 32, 32, 3) (10000, 32, 32, 3)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 32, 32, 3)	0
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650

Total params: 167,562 (654.54 KB)

Trainable params: 167,562 (654.54 KB)

Non-trainable params: 0 (0.00 B)

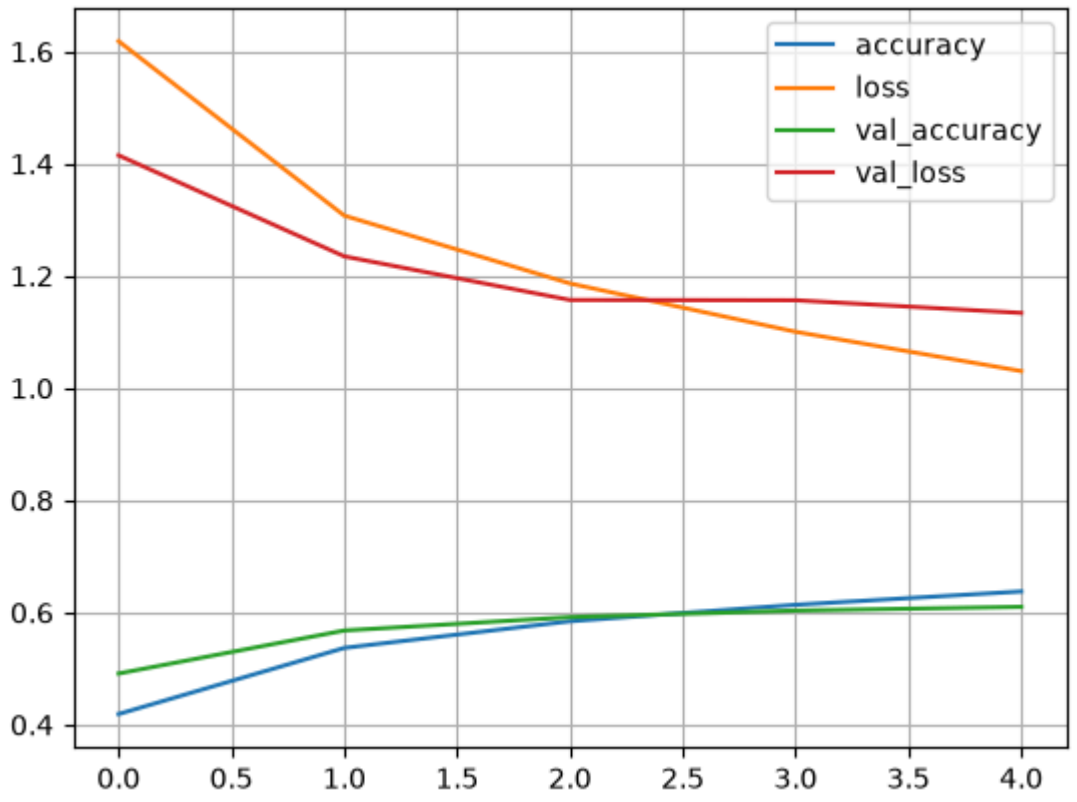
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

```
Epoch 1/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.4190 - loss: 1.6185 - val_accuracy: 0.4912 - val_loss:
1.4150
Epoch 2/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.5369 - loss: 1.3075 - val_accuracy: 0.5679 - val_loss:
1.2350
Epoch 3/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.5844 - loss: 1.1866 - val_accuracy: 0.5917 - val_loss:
1.1572
Epoch 4/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.6137 - loss: 1.1005 - val_accuracy: 0.6034 - val_loss:
1.1567
Epoch 5/5
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.6375 - loss: 1.0304 - val_accuracy: 0.6101 - val_loss:
1.1341
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
 1/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.7188 - loss: 0.9711
35/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6196 - loss: 1.1122
64/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6123 - loss: 1.1332
87/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6038 - loss: 1.1439
112/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6133 - loss: 1.1278
140/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6187 - loss: 1.1170
167/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6132 - loss: 1.1265
193/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6142 - loss: 1.1269
219/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6117 - loss: 1.1344
246/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6118 - loss: 1.1342
272/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6119 - loss: 1.1324
298/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6094 - loss: 1.1364
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6101 - loss: 1.1341
test loss: 1.1341  test accuracy: 0.6101
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


accuracy
loss
val_accuracy
val_loss
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

2B. CNN - Hand-written MNIST

CNN baseline.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/HandWritten MNIST/mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/HandWritten MNIST/mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)
x_test = test.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)

print(x_train.shape, x_test.shape)
```

```
(60000, 28, 28, 1) (10000, 28, 28, 1)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

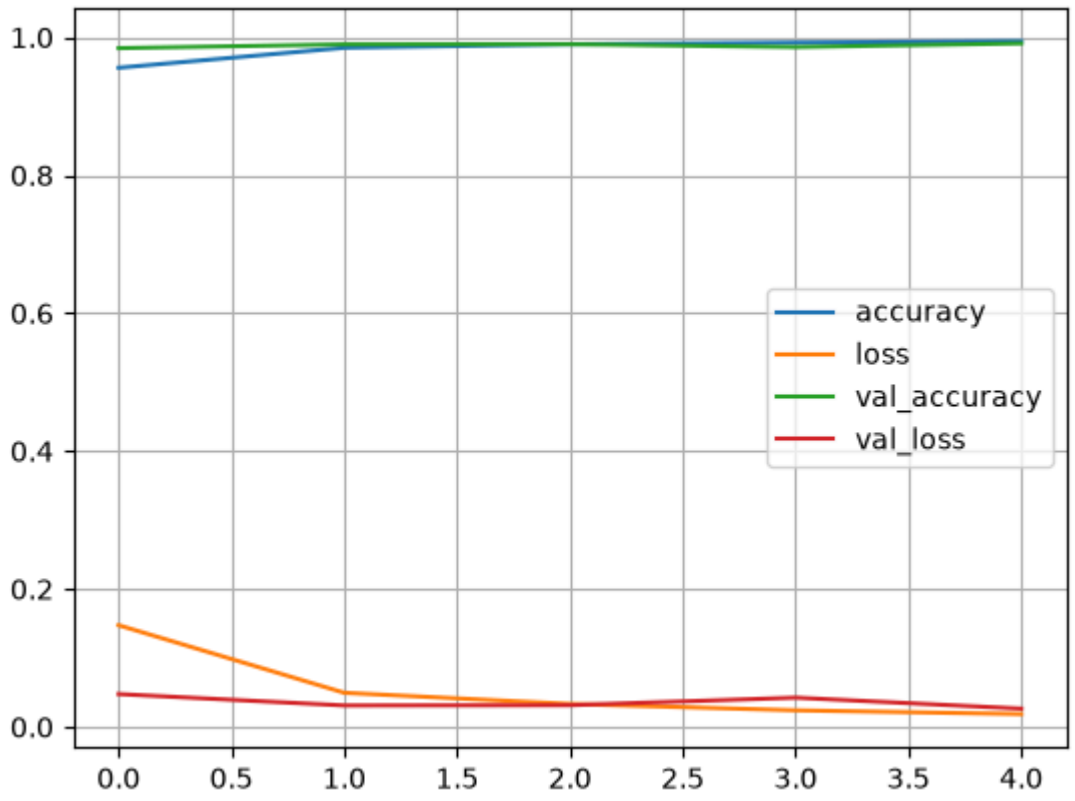
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9559 - loss: 0.1474 - val_accuracy: 0.9843 - val_loss:
0.0476
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9848 - loss: 0.0494 - val_accuracy: 0.9898 - val_loss:
0.0311
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9899 - loss: 0.0333 - val_accuracy: 0.9900 - val_loss:
0.0315
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9923 - loss: 0.0240 - val_accuracy: 0.9859 - val_loss:
0.0424
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9943 - loss: 0.0186 - val_accuracy: 0.9912 - val_loss:
0.0265
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
  1/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 1.0000 - loss: 0.0091
 45/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9875 - loss: 0.0301
 84/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9851 - loss: 0.0379
113/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9864 - loss: 0.0390
143/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9869 - loss: 0.0390
177/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9882 - loss: 0.0341
210/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9887 - loss: 0.0333
243/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9900 - loss: 0.0292
275/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9908 - loss: 0.0272
308/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9911 - loss: 0.0268
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9912 - loss: 0.0265
test loss: 0.0265  test accuracy: 0.9912
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
accuracy
loss
val_accuracy
val_loss

2C. CNN - Sign Language MNIST

CNN baseline.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)
x_test = test.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)

print(x_train.shape, x_test.shape)
```

```
(27455, 28, 28, 1) (7172, 28, 28, 1)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(25, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 25)	1,625

Total params: 122,905 (480.10 KB)

Trainable params: 122,905 (480.10 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5)
```

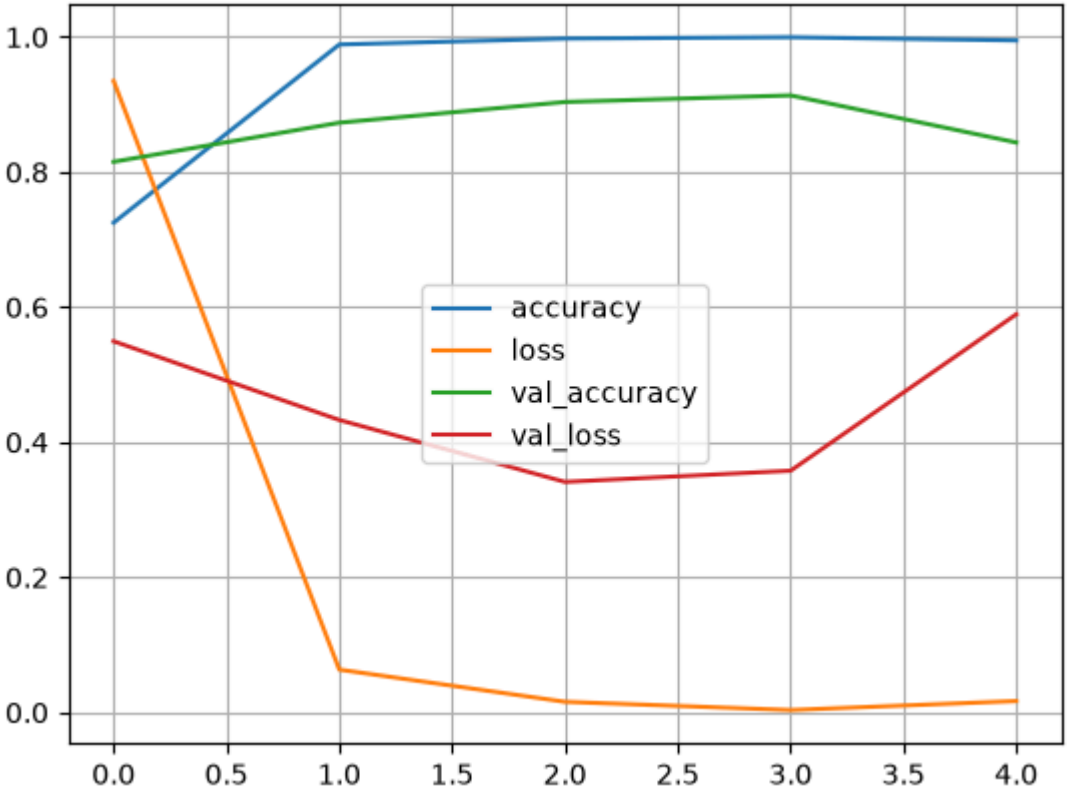
```
Epoch 1/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.7251 - loss: 0.9349 - val_accuracy: 0.8153 - val_loss: 0.5495
Epoch 2/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9889 - loss: 0.0633 - val_accuracy: 0.8731 - val_loss: 0.4329
Epoch 3/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9977 - loss: 0.0156 - val_accuracy: 0.9038 - val_loss: 0.3412
Epoch 4/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9997 - loss: 0.0034 - val_accuracy: 0.9136 - val_loss: 0.3578
Epoch 5/5
858/858 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9950 - loss: 0.0168 - val_accuracy: 0.8438 - val_loss: 0.5894
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test loss: {loss:.4f}  test accuracy: {acc:.4f}")
```

```
 1/225 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9062 - loss: 0.3210
45/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8458 - loss: 0.6060
```

```
 87/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8481 - loss: 0.5879
120/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8474 - loss: 0.5806
152/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8481 - loss: 0.5756
188/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8446 - loss: 0.5846
223/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8438 - loss: 0.5905
225/225 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8438 - loss: 0.5894
test loss: 0.5894  test accuracy: 0.8438
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



3A. Autoencoder - Credit Card Fraud

Train AE on non-fraud only. Flag high reconstruction error as fraud.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, classification_report
```

In [2]:
```
df = pd.read_csv("../datasets/AutoEncoder/CreditCard/creditcard.csv")
y = df["Class"].to_numpy()
x = df.drop(columns=["Class", "Time"]).to_numpy()

scaler = StandardScaler()
x = scaler.fit_transform(x)

x_train_norm = x[y == 0]
print("normal train:", x_train_norm.shape, "frauds:", y.sum())
```

```
normal train: (284315, 29) frauds: 492
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(29,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(29),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	480
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 16)	144
dense_3 (Dense)	(None, 29)	493

Total params: 1,253 (4.89 KB)

Trainable params: 1,253 (4.89 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train_norm, x_train_norm, epochs=5, validation_split=0.1)
```

```
Epoch 1/5
7997/7997 ━━━━━━━━━━━━━━━━━━━━ 4s 437us/step - loss: 0.5796 - val_loss: 0.5073
Epoch 2/5
7997/7997 ━━━━━━━━━━━━━━━━━━━━ 3s 432us/step - loss: 0.4708 - val_loss: 0.4747
Epoch 3/5
7997/7997 ━━━━━━━━━━━━━━━━━━━━ 3s 426us/step - loss: 0.4448 - val_loss: 0.4582
Epoch 4/5
7997/7997 ━━━━━━━━━━━━━━━━━━━━ 3s 425us/step - loss: 0.4330 - val_loss: 0.4635
Epoch 5/5
7997/7997 ━━━━━━━━━━━━━━━━━━━━ 3s 427us/step - loss: 0.4271 - val_loss: 0.4720
```

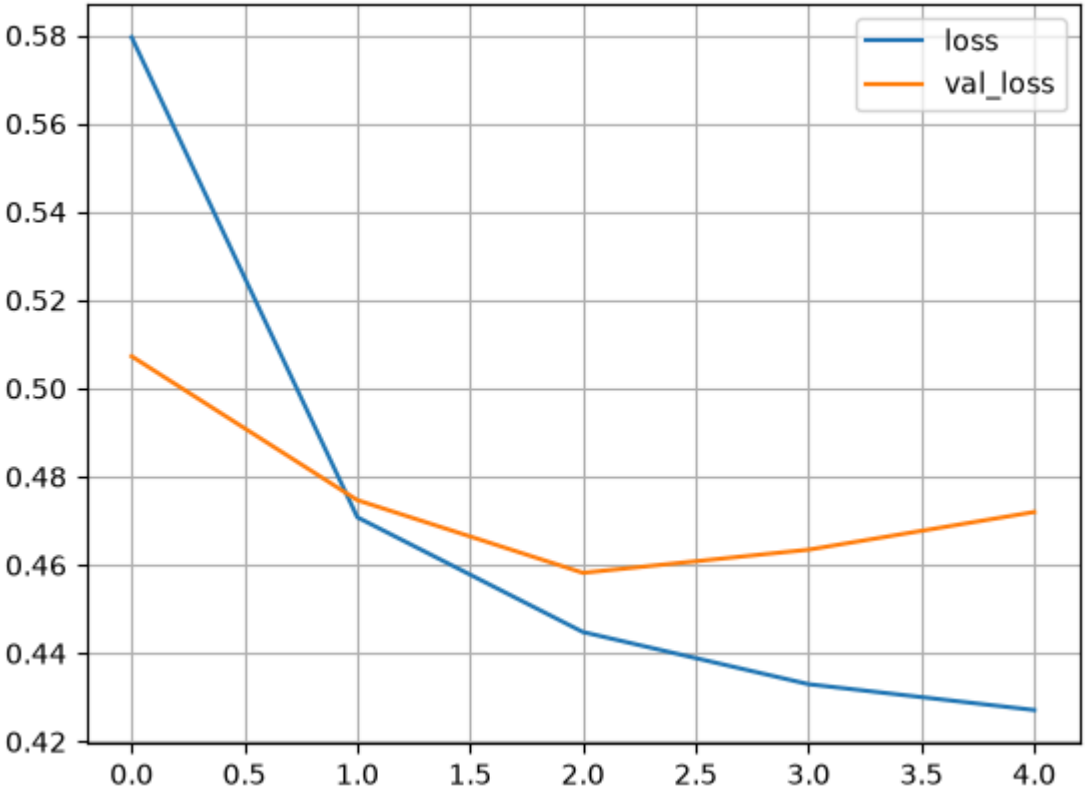
In [5]:
```
recon = model.predict(x)
err = np.mean((x - recon) ** 2, axis=1)

auc = roc_auc_score(y, err)
threshold = np.quantile(err[y == 0], 0.99)
y_pred = (err > threshold).astype(int)

print(f"AUC: {auc:.4f}")
print(classification_report(y, y_pred, target_names=["normal", "fraud"]))
```

```
AUC: 0.9405
              precision    recall  f1-score   support
      normal       1.00      0.99      0.99    284315
       fraud       0.12      0.81      0.21       492
    accuracy                           0.99    284807
   macro avg       0.56      0.90      0.60    284807
weighted avg       1.00      0.99      0.99    284807
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



3B. Autoencoder - ECG (substitute for MVTec AD)

Dense autoencoder on ECG5000. Train on normal class, detect anomalies via reconstruction error.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import roc_auc_score, classification_report
```

In [2]:
```
df = pd.read_csv("../datasets/AutoEncoder/ECG/ecg.csv", header=None)
data = df.to_numpy()

x = data[:, :-1].astype("float32")
y = (data[:, -1] != 1).astype(int)  # label 1 = normal, else anomaly

x_min = x.min()
x_max = x.max()
x = (x - x_min) / (x_max - x_min)

x_train_norm = x[y == 0]
print("normal:", x_train_norm.shape, "anomalies:", y.sum())
```

```
normal: (2919, 140) anomalies: 2079
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(140,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(140),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	9,024
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 32)	544
dense_4 (Dense)	(None, 64)	2,112
dense_5 (Dense)	(None, 140)	9,100

Total params: 23,388 (91.36 KB)
Trainable params: 23,388 (91.36 KB)
Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train_norm, x_train_norm, epochs=20, validation_split=0.1)
```

```
Epoch 1/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0435 - val_loss: 0.0011
Epoch 2/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 812us/step - loss: 0.0010 - val_loss: 9.9944e-04
Epoch 3/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 769us/step - loss: 0.0010 - val_loss: 9.7893e-04
Epoch 4/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 773us/step - loss: 0.0010 - val_loss: 9.5655e-04
Epoch 5/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 784us/step - loss: 9.6441e-04 - val_loss: 8.9663e-04
Epoch 6/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 779us/step - loss: 8.7473e-04 - val_loss: 7.7210e-04
Epoch 7/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 770us/step - loss: 7.5635e-04 - val_loss: 6.8392e-04
Epoch 8/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 773us/step - loss: 7.2000e-04 - val_loss: 6.7090e-04
Epoch 9/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 771us/step - loss: 7.0718e-04 - val_loss: 6.7581e-04
Epoch 10/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 766us/step - loss: 6.9657e-04 - val_loss: 6.8220e-04
Epoch 11/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 766us/step - loss: 6.9104e-04 - val_loss: 6.4356e-04
Epoch 12/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 766us/step - loss: 6.8257e-04 - val_loss: 6.4097e-04
Epoch 13/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 761us/step - loss: 6.6950e-04 - val_loss: 6.4121e-04
Epoch 14/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 766us/step - loss: 6.5616e-04 - val_loss: 6.1523e-04
Epoch 15/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 776us/step - loss: 6.3310e-04 - val_loss: 6.1491e-04
Epoch 16/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 761us/step - loss: 5.9405e-04 - val_loss: 5.3859e-04
Epoch 17/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 764us/step - loss: 5.4644e-04 - val_loss: 5.0260e-04
Epoch 18/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 770us/step - loss: 5.0324e-04 - val_loss: 4.6175e-04
Epoch 19/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 769us/step - loss: 4.8080e-04 - val_loss: 4.6130e-04
Epoch 20/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 766us/step - loss: 4.7742e-04 - val_loss: 4.7681e-04
```

In [5]:
```
recon = model.predict(x)
err = np.mean((x - recon) ** 2, axis=1)

auc = roc_auc_score(y, err)
threshold = np.quantile(err[y == 0], 0.95)
y_pred = (err > threshold).astype(int)

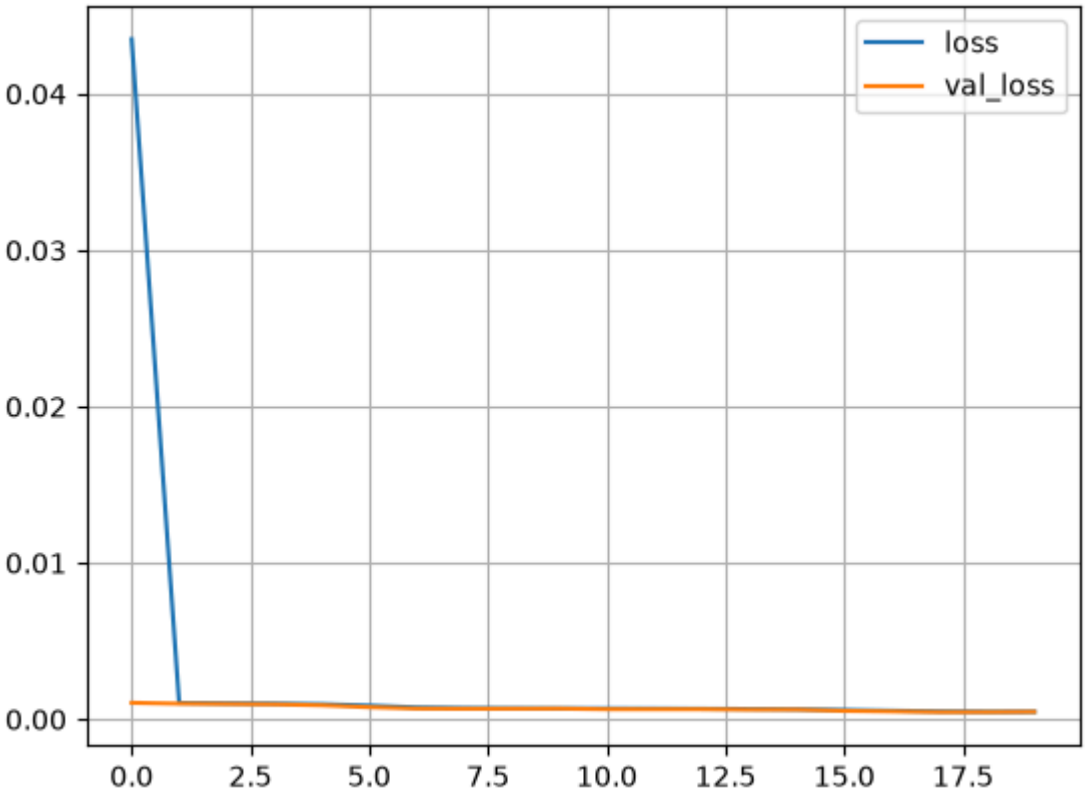
print(f"AUC: {auc:.4f}")
print(classification_report(y, y_pred, target_names=["normal", "anomaly"]))
```

```
AUC: 0.9907
              precision    recall  f1-score   support

      normal       1.00      0.95      0.97      2919
     anomaly       0.93      1.00      0.96      2079

    accuracy                           0.97      4998
   macro avg       0.97      0.97      0.97      4998
weighted avg       0.97      0.97      0.97      4998
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


loss
val_loss
0.04
0.03
0.02
0.01
0.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

3C. LSTM Autoencoder - Energy Consumption

Sliding windows -> LSTM AE -> reconstruction error.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import MinMaxScaler
```

In [2]:
```
df = pd.read_csv("../datasets/AutoEncoder/Energy Consumption/energy consumption.csv")
df = df.dropna()

y = df["Anomaly_Label"].to_numpy().astype(int)
x = df.drop(columns=["Timestamp", "Anomaly_Label"]).to_numpy().astype("float32")

scaler = MinMaxScaler()
x = scaler.fit_transform(x)

# sliding windows (W=10)
W = 10
Xw = []
yw = []
for i in range(len(x) - W + 1):
    Xw.append(x[i:i+W])
    yw.append(y[i:i+W].max())
Xw = np.array(Xw)
yw = np.array(yw)

Xw_norm = Xw[yw == 0]
print("windows:", Xw.shape, "normal:", Xw_norm.shape)
```

```
windows: (4991, 10, 5) normal: (3527, 10, 5)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(10, Xw.shape[2])),
```

```
    layers.LSTM(16, activation="relu"),
    layers.RepeatVector(10),
    layers.LSTM(16, activation="relu", return_sequences=True),
    layers.TimeDistributed(layers.Dense(Xw.shape[2])),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 16)	1,408
repeat_vector (RepeatVector)	(None, 10, 16)	0
lstm_1 (LSTM)	(None, 10, 16)	2,112
time_distributed (TimeDistributed)	(None, 10, 5)	85

Total params: 3,605 (14.08 KB)

Trainable params: 3,605 (14.08 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(Xw_norm, Xw_norm, epochs=10, validation_split=0.1)
```

```
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0921 - val_loss: 0.0292
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0216 - val_loss: 0.0172
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0166 - val_loss: 0.0160
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0160 - val_loss: 0.0159
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0159 - val_loss: 0.0157
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0158 - val_loss: 0.0156
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0157 - val_loss: 0.0156
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0156 - val_loss: 0.0155
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0155 - val_loss: 0.0152
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0153 - val_loss: 0.0151
```

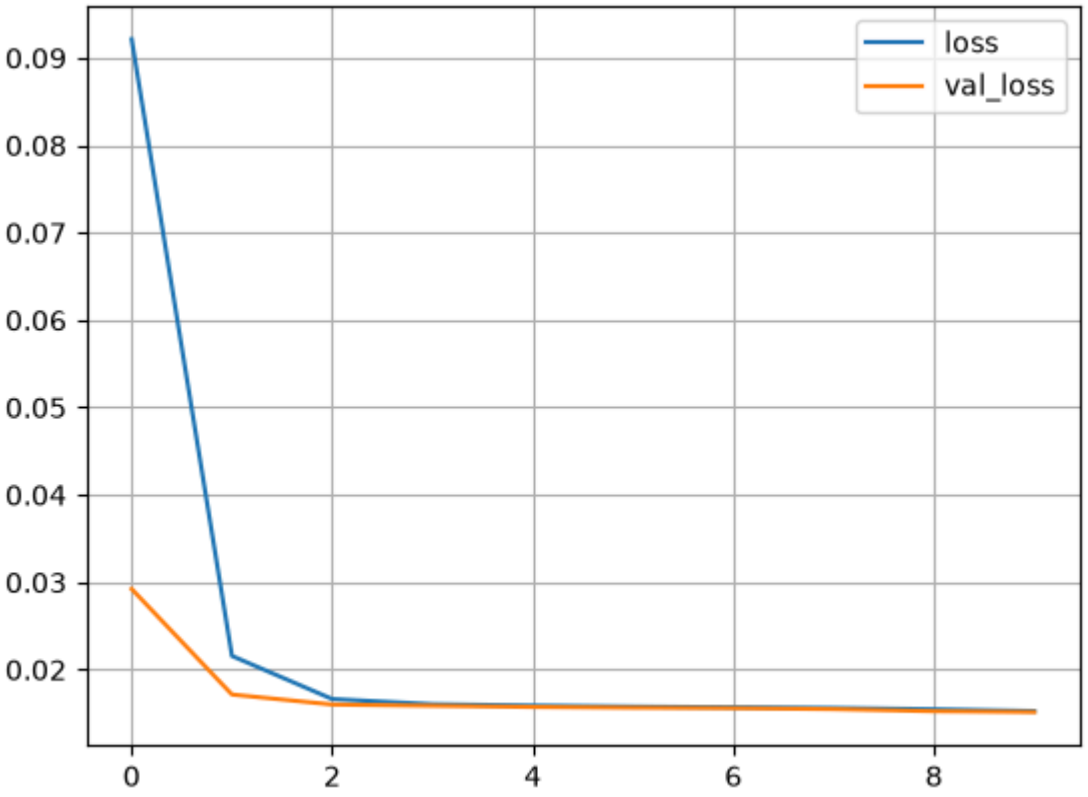
In [5]:
```
recon = model.predict(Xw)
err = np.mean((Xw - recon) ** 2, axis=(1, 2))

threshold = np.quantile(err[yw == 0], 0.95)
y_pred = (err > threshold).astype(int)

print(f"anomalies predicted: {y_pred.sum()}  actual: {yw.sum()}")
print(f"threshold: {threshold:.6f}")
```

```
anomalies predicted: 471  actual: 1464
threshold: 0.020774
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


loss
val_loss
0.09
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0
2
4
6
8

3D. LSTM Autoencoder - NASA Turbofan Engines

Train AE on first-20%-of-cycles (healthy) sensor data.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import MinMaxScaler
```

In [2]:
```
df = pd.read_csv("../datasets/AutoEncoder/NASA Turbofan ENgine/NASA TurboFan Engine.csv")

df["max_cycle"] = df.groupby("unit")["cycle"].transform("max")
df["healthy"] = df["cycle"] <= df["max_cycle"] * 0.2

feat_cols = [c for c in df.columns if c.startswith("sensor") or c.startswith("setting")]
x = df[feat_cols].to_numpy().astype("float32")

scaler = MinMaxScaler()
x = scaler.fit_transform(x)
y = (~df["healthy"]).astype(int).to_numpy()

# sliding windows (W=15)
W = 15
Xw = []
yw = []
for i in range(len(x) - W + 1):
    Xw.append(x[i:i+W])
    yw.append(y[i:i+W].max())
Xw = np.array(Xw)
yw = np.array(yw)

Xw_norm = Xw[yw == 0]
print("windows:", Xw.shape, "healthy:", Xw_norm.shape)
```

```
windows: (20617, 15, 24) healthy: (2686, 15, 24)
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(15, Xw.shape[2])),
    layers.LSTM(32, activation="relu"),
    layers.RepeatVector(15),
    layers.LSTM(32, activation="relu", return_sequences=True),
    layers.TimeDistributed(layers.Dense(Xw.shape[2])),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 32)	7,296
repeat_vector (RepeatVector)	(None, 15, 32)	0
lstm_1 (LSTM)	(None, 15, 32)	8,320
time_distributed (TimeDistributed)	(None, 15, 24)	792

Total params: 16,408 (64.09 KB)

Trainable params: 16,408 (64.09 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(Xw_norm, Xw_norm, epochs=5, validation_split=0.1)
```

```
Epoch 1/5
76/76 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0735 - val_loss: 0.0128
Epoch 2/5
76/76 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0100 - val_loss: 0.0078
Epoch 3/5
76/76 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0079 - val_loss: 0.0074
Epoch 4/5
76/76 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0076 - val_loss: 0.0073
Epoch 5/5
76/76 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0075 - val_loss: 0.0072
```

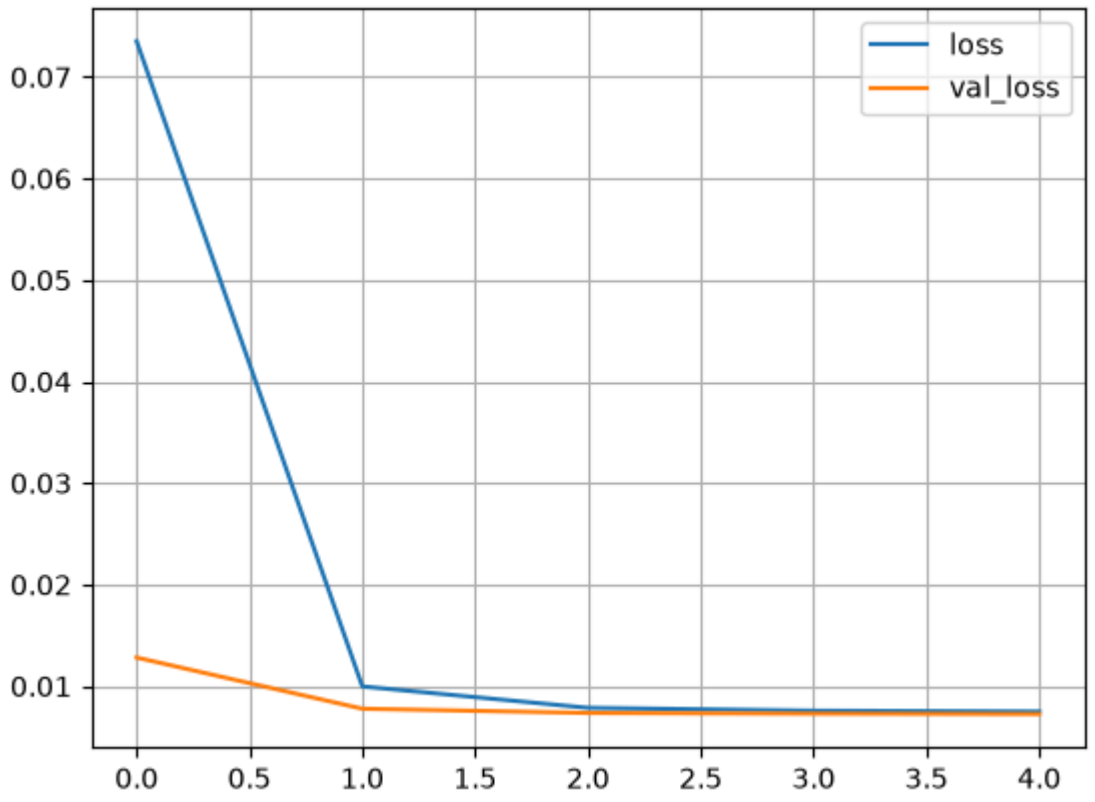
In [5]:
```
recon = model.predict(Xw)
err = np.mean((Xw - recon) ** 2, axis=(1, 2))

threshold = np.quantile(err[yw == 0], 0.95)
y_pred = (err > threshold).astype(int)

print(f"flagged faulty: {y_pred.sum()}  actual non-healthy: {yw.sum()}")
print(f"threshold: {threshold:.6f}")
```

```
flagged faulty: 4079  actual non-healthy: 17931
threshold: 0.013123
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


loss
val_loss
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

3E. Autoencoder - ECG (substitute for NSL-KDD)

Dense autoencoder on ECG5000. Train on normal class, detect anomalies via reconstruction error.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import roc_auc_score, classification_report
```

In [2]:
```
df = pd.read_csv("../datasets/AutoEncoder/ECG/ecg.csv", header=None)
data = df.to_numpy()

x = data[:, :-1].astype("float32")
y = (data[:, -1] != 1).astype(int)  # label 1 = normal, else anomaly

x_min = x.min()
x_max = x.max()
x = (x - x_min) / (x_max - x_min)

x_train_norm = x[y == 0]
print("normal:", x_train_norm.shape, "anomalies:", y.sum())
```

```
normal: (2919, 140) anomalies: 2079
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(140,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(140),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	9,024
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 32)	544
dense_4 (Dense)	(None, 64)	2,112
dense_5 (Dense)	(None, 140)	9,100

Total params: 23,388 (91.36 KB)
Trainable params: 23,388 (91.36 KB)
Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train_norm, x_train_norm, epochs=20, validation_split=0.1)
```

```
Epoch 1/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0462 - val_loss: 0.0010
Epoch 2/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 835us/step - loss: 0.0010 - val_loss: 9.8390e-04
Epoch 3/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 829us/step - loss: 0.0010 - val_loss: 9.7840e-04
Epoch 4/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 817us/step - loss: 0.0010 - val_loss: 9.7430e-04
Epoch 5/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 820us/step - loss: 0.0010 - val_loss: 9.7610e-04
Epoch 6/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 817us/step - loss: 9.9046e-04 - val_loss: 9.5851e-04
Epoch 7/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 786us/step - loss: 9.6717e-04 - val_loss: 9.4685e-04
Epoch 8/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 790us/step - loss: 9.0143e-04 - val_loss: 8.1285e-04
Epoch 9/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 780us/step - loss: 7.7681e-04 - val_loss: 6.8186e-04
Epoch 10/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 778us/step - loss: 7.0728e-04 - val_loss: 6.5678e-04
Epoch 11/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 764us/step - loss: 6.9072e-04 - val_loss: 6.4703e-04
Epoch 12/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 827us/step - loss: 6.7102e-04 - val_loss: 6.2200e-04
Epoch 13/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 822us/step - loss: 6.4953e-04 - val_loss: 6.0589e-04
Epoch 14/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 814us/step - loss: 6.1826e-04 - val_loss: 5.7961e-04
Epoch 15/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 817us/step - loss: 5.8065e-04 - val_loss: 6.0688e-04
Epoch 16/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 814us/step - loss: 5.2276e-04 - val_loss: 4.6572e-04
Epoch 17/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 805us/step - loss: 4.7715e-04 - val_loss: 4.7632e-04
Epoch 18/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 804us/step - loss: 4.6694e-04 - val_loss: 4.6752e-04
Epoch 19/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 794us/step - loss: 4.6360e-04 - val_loss: 4.4612e-04
Epoch 20/20
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 813us/step - loss: 4.6061e-04 - val_loss: 4.4485e-04
```

In [5]:
```
recon = model.predict(x)
err = np.mean((x - recon) ** 2, axis=1)

auc = roc_auc_score(y, err)
threshold = np.quantile(err[y == 0], 0.95)
y_pred = (err > threshold).astype(int)

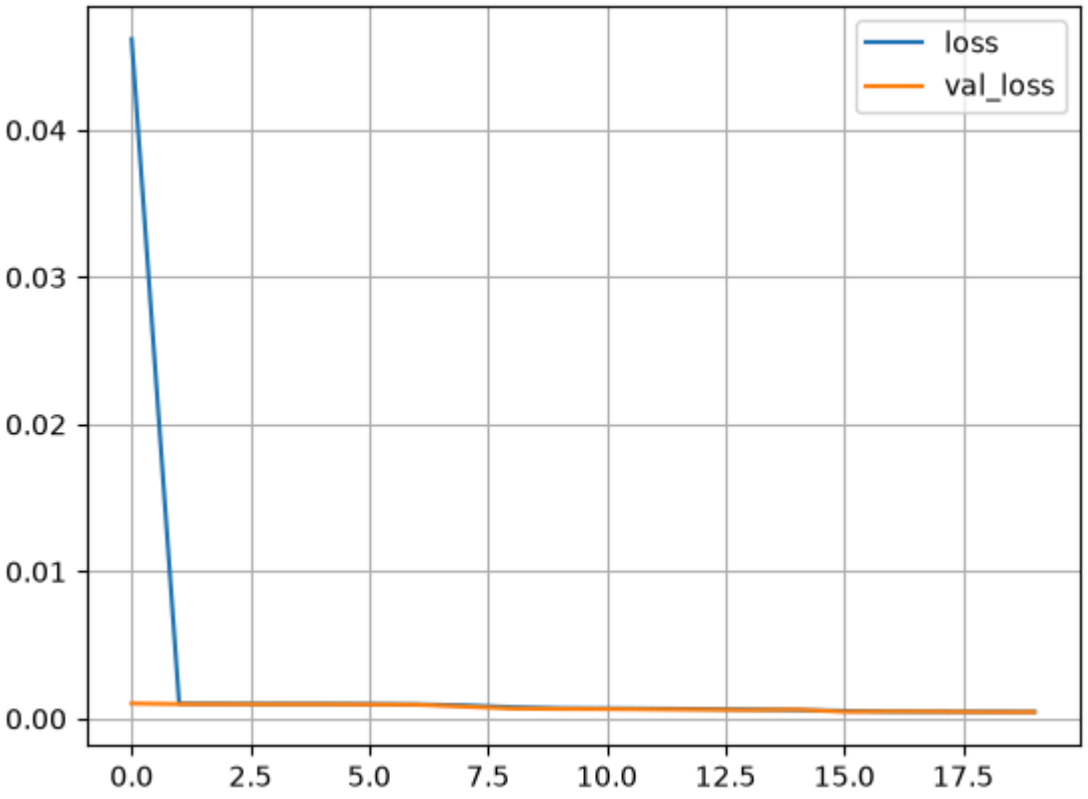
print(f"AUC: {auc:.4f}")
print(classification_report(y, y_pred, target_names=["normal", "anomaly"]))
```

```
AUC: 0.9924
              precision    recall  f1-score   support

      normal       1.00      0.95      0.97      2919
     anomaly       0.93      1.00      0.96      2079

    accuracy                           0.97      4998
   macro avg       0.97      0.97      0.97      4998
weighted avg       0.97      0.97      0.97      4998
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


loss
val_loss
0.04
0.03
0.02
0.01
0.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

3F. Autoencoder - Sign MNIST Anomaly

Treat label 0 as anomaly. Train AE on remaining 23 classes.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import roc_auc_score, classification_report
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/Sign Language MNIST/sign_mnist_test.csv")

df = pd.concat([train, test], ignore_index=True)
y_anom = (df["label"] == 0).to_numpy().astype(int)
x = df.drop(columns="label").to_numpy().astype("float32") / 255.0

x_train_norm = x[y_anom == 0]
print("normal (23 classes):", x_train_norm.shape, "anomaly (class 0):", y_anom.sum())
```

```
normal (23 classes): (33170, 784) anomaly (class 0): 1457
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(784, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="mse")
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 128)	8,320
dense_3 (Dense)	(None, 784)	101,136

Total params: 218,192 (852.31 KB)

Trainable params: 218,192 (852.31 KB)

Non-trainable params: 0 (0.00 B)

In [4]:
```
history = model.fit(x_train_norm, x_train_norm, epochs=5, validation_split=0.1)
```

```
Epoch 1/5
933/933 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0163 - val_loss: 0.0098
Epoch 2/5
933/933 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0089 - val_loss: 0.0081
Epoch 3/5
933/933 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0078 - val_loss: 0.0076
Epoch 4/5
933/933 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0072 - val_loss: 0.0070
Epoch 5/5
933/933 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0068 - val_loss: 0.0066
```

In [5]:
```
recon = model.predict(x)
err = np.mean((x - recon) ** 2, axis=1)

auc = roc_auc_score(y_anom, err)
threshold = np.quantile(err[y_anom == 0], 0.95)
y_pred = (err > threshold).astype(int)

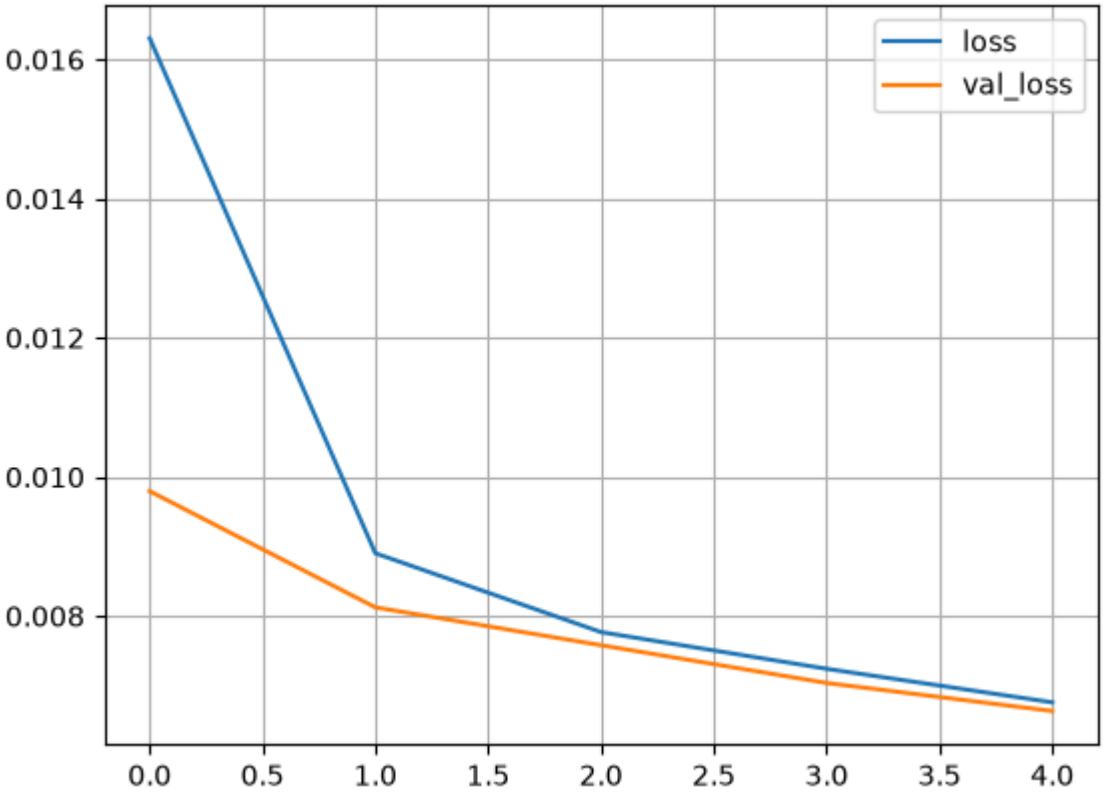
print(f"AUC: {auc:.4f}")
print(classification_report(y_anom, y_pred, target_names=["normal", "anomaly"]))
```

```
AUC: 0.6271
              precision    recall  f1-score   support

      normal       0.96      0.95      0.95     33170
     anomaly       0.07      0.08      0.08      1457

    accuracy                           0.91     34627
   macro avg       0.51      0.52      0.52     34627
weighted avg       0.92      0.91      0.92     34627
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```



4A. CBOW Word2Vec - IMDB Sentiment

Train gensim Word2Vec (sg=0 = CBOW), average embeddings per doc, classify.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re
from gensim.models import Word2Vec
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
df = pd.read_csv("../datasets/CBOW/IMDB Dataset.csv")
df = df.sample(10000, random_state=42).reset_index(drop=True)
y = (df["sentiment"] == "positive").astype(int).to_numpy()

sentences = []
for review in df["review"]:
    tokens = re.findall(r"[a-z]+", review.lower())
    sentences.append(tokens)

print("docs:", len(sentences), "example:", sentences[0][:10])
```

```
docs: 10000 example: ['i', 'really', 'liked', 'this', 'summerslam', 'due', 'to', 'the', 'look', 'of']
```

In [3]:
```
w2v = Word2Vec(sentences, sg=0)
print("vocab:", len(w2v.wv))
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
vocab: 18402
```

In [4]:
```
X = []
for sent in sentences:
    vec = w2v.wv.get_mean_vector(sent, ignore_missing=True)
    X.append(vec)
```

```
X = np.array(X)
print(X.shape)
```

```
(10000, 100)
```

In [5]:
```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

model = keras.Sequential([
    layers.Input(shape=(X.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
history = model.fit(X_tr, y_tr, validation_data=(X_te, y_te), epochs=10)
```

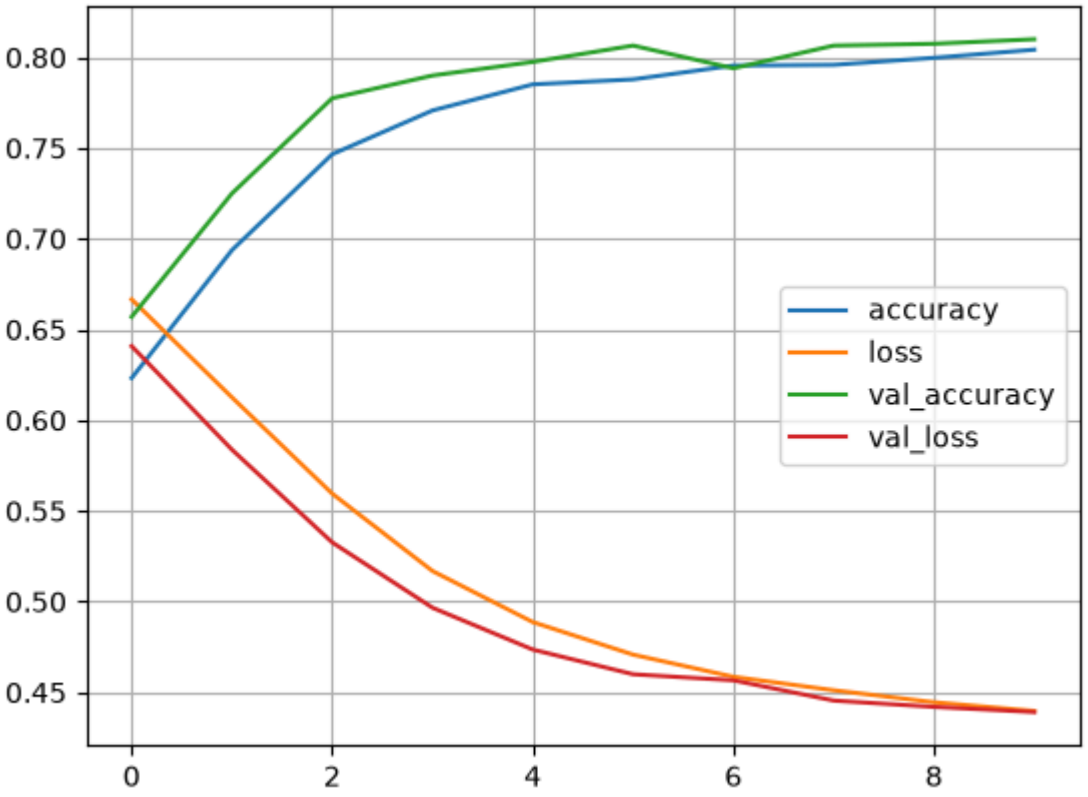
```
Epoch 1/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 2:08 518ms/step - accuracy: 0.5312 - loss: 0.6925
 99/250 ━━━━━━━━━━━━━━━━━━━━ 0s 510us/step - accuracy: 0.6001 - loss: 0.6812
230/250 ━━━━━━━━━━━━━━━━━━━━ 0s 437us/step - accuracy: 0.6178 - loss: 0.6688
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6233 - loss: 0.6666 - val_accuracy: 0.6570 - val_loss: 0.6408
Epoch 2/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8125 - loss: 0.5938
131/250 ━━━━━━━━━━━━━━━━━━━━ 0s 387us/step - accuracy: 0.6794 - loss: 0.6260
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 558us/step - accuracy: 0.6938 - loss: 0.6126 - val_accuracy: 0.7250 - val_loss:
0.5838
Epoch 3/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.6562 - loss: 0.6242
131/250 ━━━━━━━━━━━━━━━━━━━━ 0s 386us/step - accuracy: 0.7402 - loss: 0.5676
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 543us/step - accuracy: 0.7466 - loss: 0.5595 - val_accuracy: 0.7775 - val_loss:
0.5326
Epoch 4/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7812 - loss: 0.5142
131/250 ━━━━━━━━━━━━━━━━━━━━ 0s 387us/step - accuracy: 0.7646 - loss: 0.5279
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 544us/step - accuracy: 0.7707 - loss: 0.5169 - val_accuracy: 0.7900 - val_loss:
0.4966
Epoch 5/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8438 - loss: 0.5114
129/250 ━━━━━━━━━━━━━━━━━━━━ 0s 393us/step - accuracy: 0.7846 - loss: 0.4950
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 546us/step - accuracy: 0.7851 - loss: 0.4887 - val_accuracy: 0.7975 - val_loss:
0.4736
Epoch 6/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9062 - loss: 0.3688
```

```
132/250 ━━━━━━━━━━━━━━━━━━━━ 0s 382us/step - accuracy: 0.7976 - loss: 0.4647
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 542us/step - accuracy: 0.7879 - loss: 0.4708 - val_accuracy: 0.8065 - val_loss:
0.4600
Epoch 7/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8438 - loss: 0.3752
131/250 ━━━━━━━━━━━━━━━━━━━━ 0s 385us/step - accuracy: 0.7894 - loss: 0.4627
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 544us/step - accuracy: 0.7955 - loss: 0.4586 - val_accuracy: 0.7940 - val_loss:
0.4566
Epoch 8/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.6250 - loss: 0.5419
133/250 ━━━━━━━━━━━━━━━━━━━━ 0s 381us/step - accuracy: 0.7953 - loss: 0.4492
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 542us/step - accuracy: 0.7959 - loss: 0.4511 - val_accuracy: 0.8065 - val_loss:
0.4454
Epoch 9/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8438 - loss: 0.3620
130/250 ━━━━━━━━━━━━━━━━━━━━ 0s 388us/step - accuracy: 0.8000 - loss: 0.4470
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 557us/step - accuracy: 0.7997 - loss: 0.4445 - val_accuracy: 0.8075 - val_loss:
0.4420
Epoch 10/10
  1/250 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8125 - loss: 0.4067
121/250 ━━━━━━━━━━━━━━━━━━━━ 0s 417us/step - accuracy: 0.8076 - loss: 0.4391
245/250 ━━━━━━━━━━━━━━━━━━━━ 0s 412us/step - accuracy: 0.8051 - loss: 0.4389
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 589us/step - accuracy: 0.8043 - loss: 0.4397 - val_accuracy: 0.8100 - val_loss:
0.4391
```

In [6]:
```python
loss, acc = model.evaluate(X_te, y_te)
print(f"test acc: {acc:.4f}")
print("similar to 'great':", w2v.wv.most_similar("great", topn=5))
```

```
 1/63 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7500 - loss: 0.4149
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 498us/step - accuracy: 0.8100 - loss: 0.4391
test acc: 0.8100
similar to 'great': [('fine', 0.8061846494674683), ('wonderful', 0.7775288820266724), ('good', 0.7632917761802673),
('fantastic', 0.7109645009040833), ('nice', 0.7034562230110168)]
```

In [7]:
```python
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


0.80
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0
2
4
6
8
accuracy
loss
val_accuracy
val_loss

4B. CBOW Word2Vec - Twitter

Train CBOW on tweets, inspect learned embeddings.

In [1]:
```
import pandas as pd
import re
from gensim.models import Word2Vec
```

In [2]:
```
df = pd.read_csv("../datasets/CBOW/twitter_dataset.csv")
texts = df["Text"].dropna().tolist()

sentences = []
for tweet in texts:
    tokens = re.findall(r"[a-z]+", str(tweet).lower())
    sentences.append(tokens)

print("tweets:", len(sentences))
```

```
tweets: 10000
```

In [3]:
```
w2v = Word2Vec(sentences, sg=0)
print("vocab:", len(w2v.wv))
```

```
vocab: 971
```

In [4]:
```
for word in ["love", "happy", "day"]:
    if word in w2v.wv:
        print(f"\nsimilar to {word!r}:")
        for w, s in w2v.wv.most_similar(word, topn=5):
            print(f"  {w}: {s:.3f}")
```

```
similar to 'happy':
  size: 0.998
  hold: 0.998
  another: 0.998
  tree: 0.998
  clear: 0.998
similar to 'day':
  government: 0.998
  affect: 0.997
  store: 0.997
  financial: 0.997
  best: 0.997
```

4C. CBOW Word2Vec - document3.txt

Tiny corpus, so we keep `min_count=1` (otherwise vocab is empty) and raise epochs.

In [1]:
```
import re
from gensim.models import Word2Vec
```

In [2]:
```
with open("../datasets/CBOW/document3.txt") as f:
    text = f.read()

sentences = []
for raw in text.split("."):
    if raw.strip():
        tokens = re.findall(r"[a-z]+", raw.lower())
        sentences.append(tokens)

print("sentences:", len(sentences))
print(sentences[0])
```

```
sentences: 9
['the', 'speed', 'of', 'transmission', 'is', 'an', 'important', 'point', 'of', 'difference', 'between', 'the',
'two', 'viruses']
```

In [3]:
```
w2v = Word2Vec(sentences, sg=0, min_count=1, epochs=50)
print("vocab:", len(w2v.wv))
```

```
vocab: 93
```

In [4]:
```
for word in ["covid", "virus", "influenza", "transmission"]:
    if word in w2v.wv:
        print(f"\nsimilar to {word!r}:")
        for w, s in w2v.wv.most_similar(word, topn=5):
            print(f"  {w}: {s:.3f}")
```

```
similar to 'covid':
  be: 0.543
  shorter: 0.524
  for: 0.523
  the: 0.519
  to: 0.511
similar to 'virus':
  are: 0.573
  of: 0.566
  or: 0.558
  to: 0.555
  the: 0.526
similar to 'influenza':
  are: 0.554
  illness: 0.513
  virus: 0.510
  before: 0.509
  shed: 0.497
similar to 'transmission':
  specific: 0.615
  appearance: 0.584
  the: 0.575
  to: 0.551
  or: 0.527
```

4D. CBOW Recommender

Train CBOW on tweet corpus, represent each tweet as mean embedding, recommend by cosine similarity.

In [1]:
```
import numpy as np
import pandas as pd
import re
from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity
```

In [2]:
```
df = pd.read_csv("../datasets/CBOW/twitter_dataset.csv")
texts = df["Text"].dropna().tolist()

sentences = []
for tweet in texts:
    tokens = re.findall(r"[a-z]+", str(tweet).lower())
    sentences.append(tokens)

w2v = Word2Vec(sentences, sg=0)
```

In [3]:
```
doc_vecs = []
for sent in sentences:
    vec = w2v.wv.get_mean_vector(sent, ignore_missing=True)
    doc_vecs.append(vec)
doc_vecs = np.array(doc_vecs)
print("doc matrix:", doc_vecs.shape)
```

```
doc matrix: (10000, 100)
```

In [4]:
```
query = "happy day great"
q_tokens = re.findall(r"[a-z]+", query.lower())
q_vec = w2v.wv.get_mean_vector(q_tokens, ignore_missing=True).reshape(1, -1)

sims = cosine_similarity(q_vec, doc_vecs).flatten()
top = sims.argsort()[::-1][:5]

print(f"query: {query!r}\n")
```

```python
for i in top:
    print(f"  [{sims[i]:.3f}] {texts[i][:120]}")
```

```
query: 'happy day great'
  [1.000] Week image may race. Modern prevent floor one economy local born. Land day have protect.
Force level great car care audi
  [1.000] Guy next among medical. Traditional all hard relationship. Modern direction left responsibility action.
They federal han
  [1.000] Purpose would purpose suddenly every religious. Few its already walk.
Perhaps sometimes it example according law single.
  [1.000] Compare American exist its follow bring concern significant. Direction produce family actually.
Page but across generati
  [1.000] Since baby certainly interest budget whole. Prepare paper subject sing TV.
Partner improve open movement thousand part.
```

5A. Transfer Learning - Caltech-101 (VGG16)

In [1]:
```
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = keras.utils.image_dataset_from_directory(
    "../datasets/Object Detection/caltech-101",
    image_size=(128, 128), validation_split=0.2, subset="training", seed=42)
val = keras.utils.image_dataset_from_directory(
    "../datasets/Object Detection/caltech-101",
    image_size=(128, 128), validation_split=0.2, subset="validation", seed=42)

n_classes = len(train.class_names)
print("classes:", n_classes)

train = train.prefetch(2)
val = val.prefetch(2)
```

```
Found 9144 files belonging to 102 classes.
Using 7316 files for training.
Found 9144 files belonging to 102 classes.
Using 1828 files for validation.
classes: 102
```

In [3]:
```
base = keras.applications.VGG16(include_top=False, input_shape=(128, 128, 3))
base.trainable = False

model = keras.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Rescaling(1./255),
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dense(n_classes, activation="softmax"),
])
```

```
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
vgg16 (Functional)	(None, 4, 4, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dense_1 (Dense)	(None, 102)	13,158

Total params: 14,793,510 (56.43 MB)

Trainable params: 78,822 (307.90 KB)

Non-trainable params: 14,714,688 (56.13 MB)

In [4]:
```
history = model.fit(train, validation_data=val, epochs=3)
```

```
Epoch 1/3
229/229 ━━━━━━━━━━━━━━━━━━━━ 295s 1s/step - accuracy: 0.3435 - loss: 3.1391 - val_accuracy: 0.5071 - val_loss:
2.2613
Epoch 2/3
229/229 ━━━━━━━━━━━━━━━━━━━━ 964s 4s/step - accuracy: 0.5854 - loss: 1.8434 - val_accuracy: 0.6439 - val_loss:
1.5837
Epoch 3/3
229/229 ━━━━━━━━━━━━━━━━━━━━ 1433s 6s/step - accuracy: 0.6838 - loss: 1.3465 - val_accuracy: 0.6909 - val_loss:
1.3196
```

In [5]:
```
loss, acc = model.evaluate(val)
print(f"val acc (frozen base): {acc:.4f}")
```

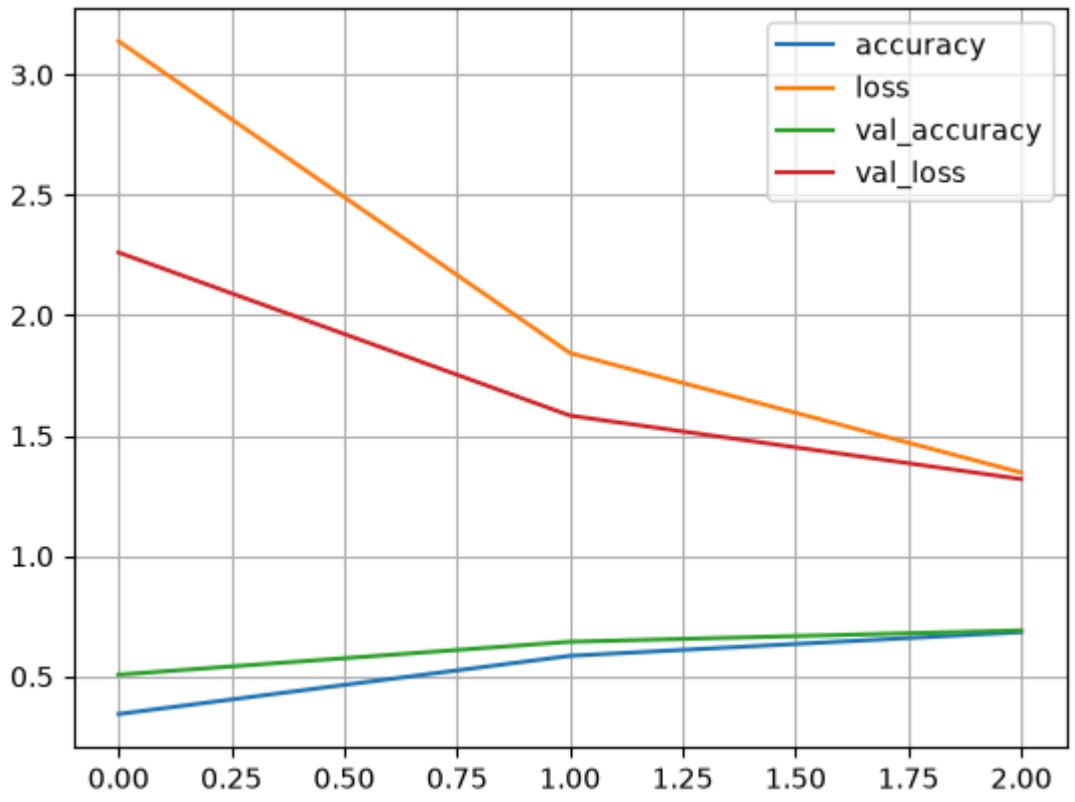
```
1/58 ━━━━━━━━━━━━━━━━━━━━ 59s 1s/step - accuracy: 0.6562 - loss: 1.3515
2/58 ━━━━━━━━━━━━━━━━━━━━ 57s 1s/step - accuracy: 0.6875 - loss: 1.3013
3/58 ━━━━━━━━━━━━━━━━━━━━ 55s 1s/step - accuracy: 0.6875 - loss: 1.3076
4/58 ━━━━━━━━━━━━━━━━━━━━ 54s 1s/step - accuracy: 0.6406 - loss: 1.4078
```

```
 5/58 ━━━━━━━━━━━━━━━━━━━━ 54s 1s/step - accuracy: 0.6250 - loss: 1.4636
 6/58 ━━━━━━━━━━━━━━━━━━━━ 53s 1s/step - accuracy: 0.6406 - loss: 1.4404
 7/58 ━━━━━━━━━━━━━━━━━━━━ 52s 1s/step - accuracy: 0.6562 - loss: 1.3828
 8/58 ━━━━━━━━━━━━━━━━━━━━ 51s 1s/step - accuracy: 0.6445 - loss: 1.4096
 9/58 ━━━━━━━━━━━━━━━━━━━━ 50s 1s/step - accuracy: 0.6562 - loss: 1.3906
10/58 ━━━━━━━━━━━━━━━━━━━━ 49s 1s/step - accuracy: 0.6750 - loss: 1.3459
11/58 ━━━━━━━━━━━━━━━━━━━━ 48s 1s/step - accuracy: 0.6790 - loss: 1.3356
12/58 ━━━━━━━━━━━━━━━━━━━━ 47s 1s/step - accuracy: 0.6797 - loss: 1.3304
13/58 ━━━━━━━━━━━━━━━━━━━━ 46s 1s/step - accuracy: 0.6875 - loss: 1.3251
14/58 ━━━━━━━━━━━━━━━━━━━━ 45s 1s/step - accuracy: 0.6786 - loss: 1.3408
15/58 ━━━━━━━━━━━━━━━━━━━━ 44s 1s/step - accuracy: 0.6771 - loss: 1.3434
16/58 ━━━━━━━━━━━━━━━━━━━━ 43s 1s/step - accuracy: 0.6797 - loss: 1.3251
17/58 ━━━━━━━━━━━━━━━━━━━━ 42s 1s/step - accuracy: 0.6746 - loss: 1.3264
18/58 ━━━━━━━━━━━━━━━━━━━━ 41s 1s/step - accuracy: 0.6840 - loss: 1.3048
19/58 ━━━━━━━━━━━━━━━━━━━━ 40s 1s/step - accuracy: 0.6743 - loss: 1.3354
20/58 ━━━━━━━━━━━━━━━━━━━━ 39s 1s/step - accuracy: 0.6781 - loss: 1.3372
21/58 ━━━━━━━━━━━━━━━━━━━━ 38s 1s/step - accuracy: 0.6830 - loss: 1.3241
22/58 ━━━━━━━━━━━━━━━━━━━━ 37s 1s/step - accuracy: 0.6832 - loss: 1.3340
23/58 ━━━━━━━━━━━━━━━━━━━━ 36s 1s/step - accuracy: 0.6834 - loss: 1.3388
24/58 ━━━━━━━━━━━━━━━━━━━━ 35s 1s/step - accuracy: 0.6797 - loss: 1.3531
25/58 ━━━━━━━━━━━━━━━━━━━━ 34s 1s/step - accuracy: 0.6800 - loss: 1.3514
26/58 ━━━━━━━━━━━━━━━━━━━━ 33s 1s/step - accuracy: 0.6767 - loss: 1.3539
27/58 ━━━━━━━━━━━━━━━━━━━━ 32s 1s/step - accuracy: 0.6782 - loss: 1.3453
28/58 ━━━━━━━━━━━━━━━━━━━━ 31s 1s/step - accuracy: 0.6842 - loss: 1.3278
29/58 ━━━━━━━━━━━━━━━━━━━━ 30s 1s/step - accuracy: 0.6843 - loss: 1.3205
30/58 ━━━━━━━━━━━━━━━━━━━━ 29s 1s/step - accuracy: 0.6875 - loss: 1.3202
31/58 ━━━━━━━━━━━━━━━━━━━━ 28s 1s/step - accuracy: 0.6885 - loss: 1.3351
32/58 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step - accuracy: 0.6875 - loss: 1.3331
33/58 ━━━━━━━━━━━━━━━━━━━━ 26s 1s/step - accuracy: 0.6866 - loss: 1.3383
34/58 ━━━━━━━━━━━━━━━━━━━━ 25s 1s/step - accuracy: 0.6847 - loss: 1.3450
35/58 ━━━━━━━━━━━━━━━━━━━━ 24s 1s/step - accuracy: 0.6866 - loss: 1.3374
36/58 ━━━━━━━━━━━━━━━━━━━━ 23s 1s/step - accuracy: 0.6884 - loss: 1.3291
37/58 ━━━━━━━━━━━━━━━━━━━━ 22s 1s/step - accuracy: 0.6875 - loss: 1.3298
38/58 ━━━━━━━━━━━━━━━━━━━━ 23s 1s/step - accuracy: 0.6891 - loss: 1.3324
39/58 ━━━━━━━━━━━━━━━━━━━━ 21s 1s/step - accuracy: 0.6915 - loss: 1.3284
40/58 ━━━━━━━━━━━━━━━━━━━━ 20s 1s/step - accuracy: 0.6898 - loss: 1.3329
41/58 ━━━━━━━━━━━━━━━━━━━━ 18s 1s/step - accuracy: 0.6883 - loss: 1.3402
42/58 ━━━━━━━━━━━━━━━━━━━━ 17s 1s/step - accuracy: 0.6875 - loss: 1.3468
43/58 ━━━━━━━━━━━━━━━━━━━━ 16s 1s/step - accuracy: 0.6875 - loss: 1.3486
44/58 ━━━━━━━━━━━━━━━━━━━━ 15s 1s/step - accuracy: 0.6875 - loss: 1.3499
45/58 ━━━━━━━━━━━━━━━━━━━━ 14s 1s/step - accuracy: 0.6861 - loss: 1.3539
```

```
46/58 ━━━━━━━━━━━━━━━━━━━━ 13s 1s/step - accuracy: 0.6848 - loss: 1.3557
47/58 ━━━━━━━━━━━━━━━━━━━━ 12s 1s/step - accuracy: 0.6862 - loss: 1.3504
48/58 ━━━━━━━━━━━━━━━━━━━━ 11s 1s/step - accuracy: 0.6901 - loss: 1.3392
49/58 ━━━━━━━━━━━━━━━━━━━━ 10s 1s/step - accuracy: 0.6901 - loss: 1.3401
50/58 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.6913 - loss: 1.3378
51/58 ━━━━━━━━━━━━━━━━━━━━ 7s 1s/step - accuracy: 0.6900 - loss: 1.3390
52/58 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.6923 - loss: 1.3328
53/58 ━━━━━━━━━━━━━━━━━━━━ 5s 1s/step - accuracy: 0.6922 - loss: 1.3269
54/58 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - accuracy: 0.6927 - loss: 1.3226
55/58 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - accuracy: 0.6920 - loss: 1.3249
56/58 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - accuracy: 0.6908 - loss: 1.3244
57/58 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.6908 - loss: 1.3212
58/58 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.6909 - loss: 1.3196
58/58 ━━━━━━━━━━━━━━━━━━━━ 64s 1s/step - accuracy: 0.6909 - loss: 1.3196
val acc (frozen base): 0.6909
```

In [6]:

```python
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


accuracy
loss
val_accuracy
val_loss
3.0
2.5
2.0
1.5
1.0
0.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

5B. Transfer Learning - Intel Image (MobileNetV2)

In [1]:
```
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
BASE = "../datasets/Object Detection/Intel Image Classification"
train = keras.utils.image_dataset_from_directory(
    BASE, image_size=(128, 128), validation_split=0.2, subset="training", seed=42)
val = keras.utils.image_dataset_from_directory(
    BASE, image_size=(128, 128), validation_split=0.2, subset="validation", seed=42)

n_classes = len(train.class_names)
print("classes:", train.class_names)

train = train.prefetch(2)
val = val.prefetch(2)
```

```
Found 12632 files belonging to 6 classes.
Using 10106 files for training.
Found 12632 files belonging to 6 classes.
Using 2526 files for validation.
classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
```

In [3]:
```
base = keras.applications.MobileNetV2(include_top=False, input_shape=(128, 128, 3))
base.trainable = False

model = keras.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Rescaling(1./127.5, offset=-1),
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(64, activation="relu"),
    layers.Dense(n_classes, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 64)	81,984
dense_1 (Dense)	(None, 6)	390

Total params: 2,340,358 (8.93 MB)

Trainable params: 82,374 (321.77 KB)

Non-trainable params: 2,257,984 (8.61 MB)

In [4]:
```
history = model.fit(train, validation_data=val, epochs=3)
```

```
Epoch 1/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 30s 89ms/step - accuracy: 0.8862 - loss: 0.3126 - val_accuracy: 0.9121 - val_loss:
0.2465
Epoch 2/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 28s 89ms/step - accuracy: 0.9287 - loss: 0.1979 - val_accuracy: 0.9153 - val_loss:
0.2350
Epoch 3/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 139s 442ms/step - accuracy: 0.9437 - loss: 0.1499 - val_accuracy: 0.9137 - val_loss:
0.2507
```

In [5]:
```
loss, acc = model.evaluate(val)
print(f"val acc: {acc:.4f}")
```

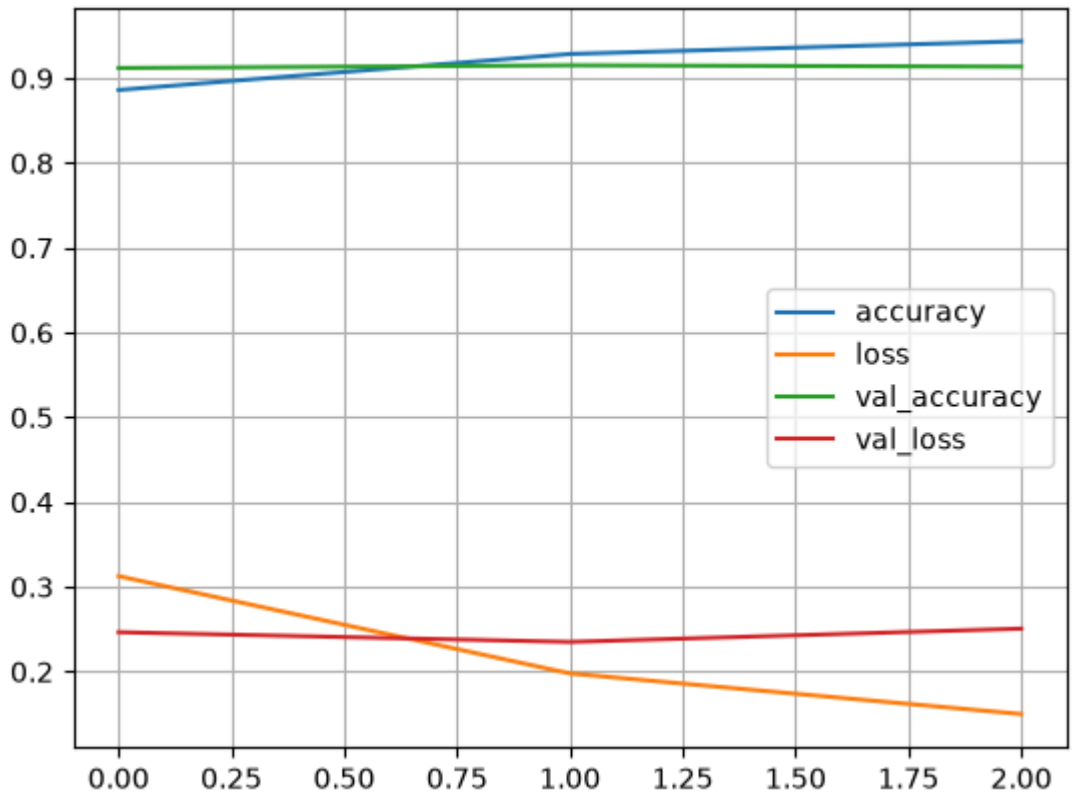
```
1/79 ━━━━━━━━━━━━━━━━━━━━ 5s 70ms/step - accuracy: 0.8438 - loss: 0.3226
2/79 ━━━━━━━━━━━━━━━━━━━━ 5s 76ms/step - accuracy: 0.9219 - loss: 0.1826
3/79 ━━━━━━━━━━━━━━━━━━━━ 5s 74ms/step - accuracy: 0.9375 - loss: 0.1862
4/79 ━━━━━━━━━━━━━━━━━━━━ 5s 71ms/step - accuracy: 0.9219 - loss: 0.2291
5/79 ━━━━━━━━━━━━━━━━━━━━ 5s 70ms/step - accuracy: 0.9187 - loss: 0.2457
6/79 ━━━━━━━━━━━━━━━━━━━━ 5s 71ms/step - accuracy: 0.9062 - loss: 0.2560
```

```
 7/79 ━━━━━━━━━━━━━━━━━━━━ 5s 72ms/step - accuracy: 0.9152 - loss: 0.2401
 8/79 ━━━━━━━━━━━━━━━━━━━━ 5s 71ms/step - accuracy: 0.9180 - loss: 0.2254
 9/79 ━━━━━━━━━━━━━━━━━━━━ 4s 71ms/step - accuracy: 0.9236 - loss: 0.2111
10/79 ━━━━━━━━━━━━━━━━━━━━ 4s 71ms/step - accuracy: 0.9219 - loss: 0.2099
11/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9233 - loss: 0.2077
12/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9219 - loss: 0.2029
13/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9183 - loss: 0.2087
14/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9174 - loss: 0.2058
15/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9167 - loss: 0.2075
16/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9180 - loss: 0.2033
17/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9210 - loss: 0.1988
18/79 ━━━━━━━━━━━━━━━━━━━━ 4s 69ms/step - accuracy: 0.9236 - loss: 0.1957
19/79 ━━━━━━━━━━━━━━━━━━━━ 4s 70ms/step - accuracy: 0.9243 - loss: 0.1927
20/79 ━━━━━━━━━━━━━━━━━━━━ 4s 69ms/step - accuracy: 0.9219 - loss: 0.2041
21/79 ━━━━━━━━━━━━━━━━━━━━ 4s 69ms/step - accuracy: 0.9211 - loss: 0.2030
22/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9190 - loss: 0.2084
23/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9198 - loss: 0.2098
24/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9193 - loss: 0.2118
25/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9162 - loss: 0.2237
26/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9123 - loss: 0.2383
27/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9109 - loss: 0.2405
28/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9141 - loss: 0.2333
29/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9149 - loss: 0.2290
30/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9177 - loss: 0.2222
31/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9204 - loss: 0.2159
32/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9189 - loss: 0.2167
33/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9195 - loss: 0.2177
34/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9173 - loss: 0.2271
35/79 ━━━━━━━━━━━━━━━━━━━━ 3s 69ms/step - accuracy: 0.9170 - loss: 0.2240
36/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9149 - loss: 0.2318
37/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9147 - loss: 0.2303
38/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9145 - loss: 0.2281
39/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9151 - loss: 0.2284
40/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9148 - loss: 0.2302
41/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9162 - loss: 0.2273
42/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9144 - loss: 0.2321
43/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9135 - loss: 0.2342
44/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9134 - loss: 0.2344
45/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9146 - loss: 0.2335
46/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9158 - loss: 0.2302
47/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9169 - loss: 0.2277
```

```
48/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9173 - loss: 0.2280
49/79 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.9158 - loss: 0.2322
50/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9144 - loss: 0.2343
51/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9148 - loss: 0.2332
52/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9147 - loss: 0.2406
53/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9151 - loss: 0.2409
54/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9161 - loss: 0.2379
55/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9153 - loss: 0.2375
56/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9141 - loss: 0.2420
57/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9134 - loss: 0.2447
58/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9122 - loss: 0.2488
59/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9131 - loss: 0.2482
60/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9130 - loss: 0.2513
61/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9139 - loss: 0.2483
62/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9108 - loss: 0.2581
63/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9122 - loss: 0.2546
64/79 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.9131 - loss: 0.2529
65/79 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - accuracy: 0.9135 - loss: 0.2518
66/79 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - accuracy: 0.9129 - loss: 0.2547
67/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9132 - loss: 0.2547
68/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9141 - loss: 0.2524
69/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9139 - loss: 0.2515
70/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9147 - loss: 0.2488
71/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9124 - loss: 0.2547
72/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9123 - loss: 0.2556
73/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9114 - loss: 0.2576
74/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9109 - loss: 0.2592
75/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9112 - loss: 0.2576
76/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9116 - loss: 0.2554
77/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9127 - loss: 0.2532
78/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9135 - loss: 0.2510
79/79 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9137 - loss: 0.2507
79/79 ━━━━━━━━━━━━━━━━━━━━ 6s 70ms/step - accuracy: 0.9137 - loss: 0.2507
val acc: 0.9137
```

In [6]:

```python
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
accuracy
loss
val_accuracy
val_loss

5C. Transfer Learning - Fashion MNIST (MobileNetV2)

Upscale 28x28 gray -> 32x32 RGB inside the model.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

In [2]:
```
train = pd.read_csv("../datasets/FNN and CNN/Fashion MNIST/fashion-mnist_train.csv")
test = pd.read_csv("../datasets/FNN and CNN/Fashion MNIST/fashion-mnist_test.csv")

y_train = train["label"].to_numpy()
y_test = test["label"].to_numpy()
x_train = train.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)
x_test = test.drop(columns="label").to_numpy().reshape(-1, 28, 28, 1)

# subsample to keep it fast
x_train = x_train[:10000]
y_train = y_train[:10000]
x_test = x_test[:2000]
y_test = y_test[:2000]

print(x_train.shape)
```

```
(10000, 28, 28, 1)
```

In [3]:
```
base = keras.applications.MobileNetV2(include_top=False, input_shape=(32, 32, 3), alpha=0.35)
base.trainable = False

inputs = keras.Input(shape=(28, 28, 1))
x = layers.Resizing(32, 32)(inputs)
x = layers.Concatenate()([x, x, x])
x = layers.Rescaling(1./127.5, offset=-1)(x)
x = base(x)
x = layers.GlobalAveragePooling2D()(x)
```

```
x = layers.Dense(64, activation="relu")(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

```
/var/folders/r0/_9rxdrf50qx7twh2z0g5921w0000gn/T/ipykernel_95449/1150300068.py:1: UserWarning: `input_shape` is
undefined or non-square, or `rows` is not in [96, 128, 160, 192, 224]. Weights for input shape (224, 224) will be
loaded as the default.
  base = keras.applications.MobileNetV2(include_top=False, input_shape=(32, 32, 3), alpha=0.35)
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0	–
resizing (Resizing)	(None, 32, 32, 1)	0	input_layer_1[0]…
concatenate (Concatenate)	(None, 32, 32, 3)	0	resizing[0][0], resizing[0][0], resizing[0][0]
rescaling (Rescaling)	(None, 32, 32, 3)	0	concatenate[0][0]
mobilenetv2_0.35_2… (Functional)	(None, 1, 1, 1280)	410,208	rescaling[0][0]
global_average_poo… (GlobalAveragePool…	(None, 1280)	0	mobilenetv2_0.35…
dense (Dense)	(None, 64)	81,984	global_average_p…
dense_1 (Dense)	(None, 10)	650	dense[0][0]

Total params: 492,842 (1.88 MB)

Trainable params: 82,634 (322.79 KB)

Non-trainable params: 410,208 (1.56 MB)

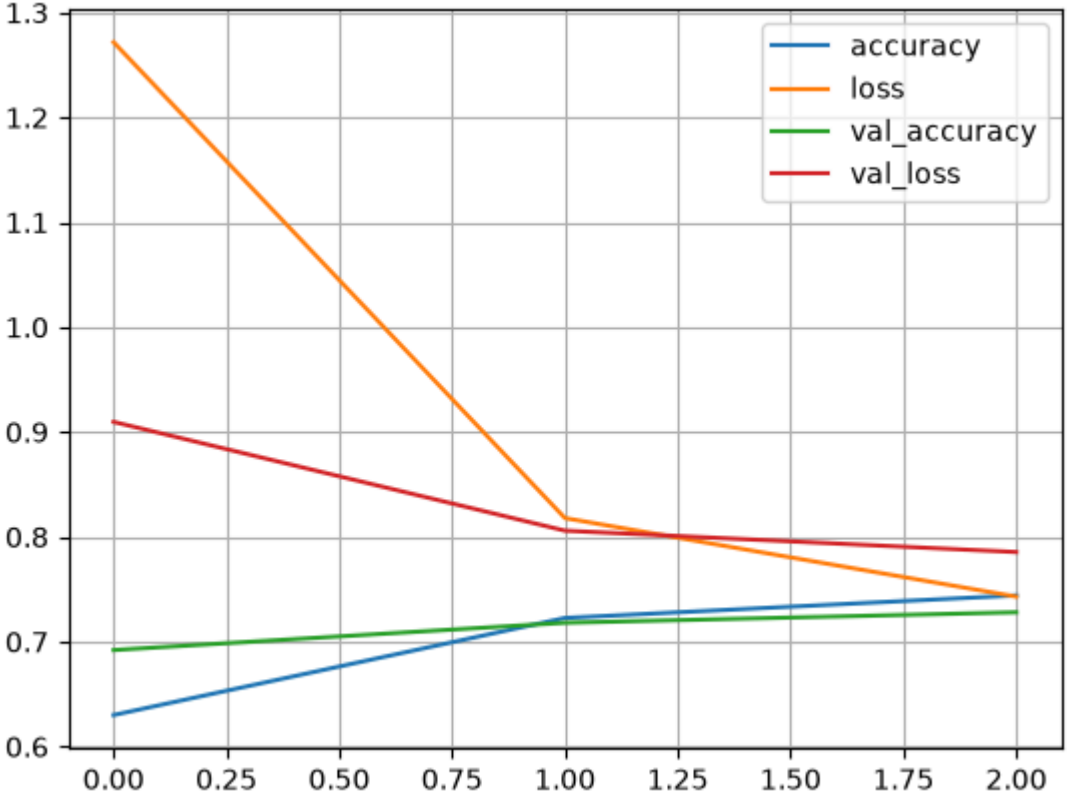
In [4]:
```
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=3)
```

```
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - accuracy: 0.6300 - loss: 1.2718 - val_accuracy: 0.6920 - val_loss: 0.9096
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.7226 - loss: 0.8179 - val_accuracy: 0.7180 - val_loss: 0.8059
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.7441 - loss: 0.7429 - val_accuracy: 0.7280 - val_loss: 0.7855
```

In [5]:
```
loss, acc = model.evaluate(x_test, y_test)
print(f"test acc: {acc:.4f}")
```

```
 1/63 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6562 - loss: 0.9779
12/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7005 - loss: 0.7801
23/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7405 - loss: 0.7260
34/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7426 - loss: 0.7316
45/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7333 - loss: 0.7744
56/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7338 - loss: 0.7743
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7280 - loss: 0.7855
test acc: 0.7280
```

In [6]:
```
for k, v in history.history.items():
    plt.plot(v, label=k)
plt.legend()
plt.grid()
plt.show()
```


accuracy
loss
val_accuracy
val_loss
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00